

Final_Project

Johann R

2026-03-10

Final Project Workdoc

Looking at the data

```
library(car)
```

```
## Loading required package: carData
```

```
library(MuMIn)
```

So far this is setup / “Data cleaning and exploratory” assignment

First step is loading in the data: setwd(“C:/Users/johan/Desktop/Le College/Q Sci 482/Final Project”)

```
# Load Data  
ungulates = read.csv("cleaned_ecology_dataset_9.csv")
```

```
# Get an initial look at the data  
head(ungulates, 10)
```

```
##           ndvi dist_water elevation human_footprint snow_depth region season  
## 1  0.6481185  1.5915271 1120.2564      86.463142   21.882679 Central Winter  
## 2  0.8452634  4.5464169 1394.5372      55.036525    3.198445 Central Summer  
## 3  0.3159613  0.1095153  442.1242      91.615872   51.858680 Central Summer  
## 4  0.6437102  0.3767398  456.8150      45.273182   22.284056 Central Winter  
## 5  0.7229648  3.9307225  929.4815      61.254256   29.994070   South Winter  
## 6  0.7916754  1.1748960 2091.0933      36.125332   25.243537 Central Summer  
## 7  0.5729577  2.3560213 1612.1296      42.558284   35.955066   South Summer  
## 8  0.3497887  0.9520071 1446.3149      48.355380   34.933924   South Winter  
## 9  0.3535932  4.5715543 1976.3815        3.203257   35.960957   South Summer  
## 10 0.7490605  4.3386752  530.6121      70.052883    8.068666 Central Winter  
##           density  
## 1  11.42866  
## 2  18.71176  
## 3  13.44218  
## 4  11.67938  
## 5  10.86497  
## 6  18.32689
```

```
## 7  16.01910
## 8  10.58822
## 9  13.05950
## 10 11.63880
```

To me, seems that *density* is the obvious response variable.

A simple research question is: “What variables are the most useful predictors for the density of ungulate populations?”

Running some linearity tests

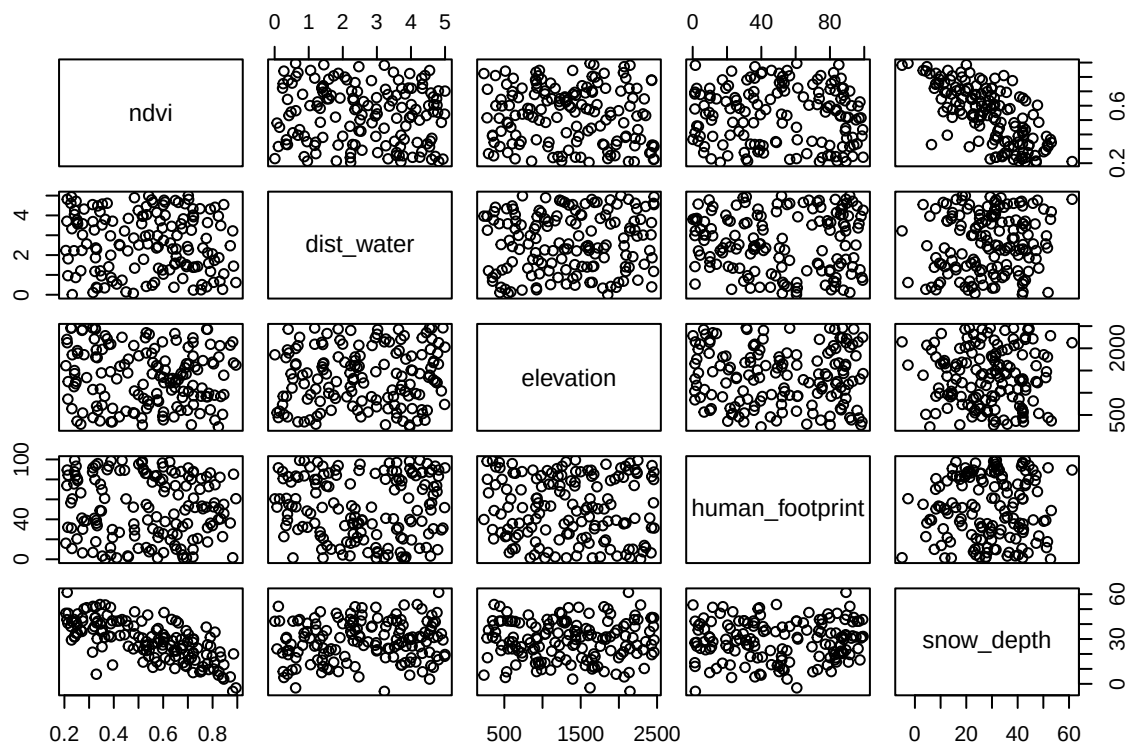
Not sure right now what “Descriptive statistucs” are supposed to be.

Can’t plot all the variables against each other in one plot, but you could run a `pairs()` funct.

Problem there is that *region* and *season* are categorical variables. Will need to deal with them later.

Taking inspiration from Lab 7. Omitting 'region' and 'season'

```
pairs(ungulates[, c("ndvi", "dist_water", "elevation", "human_footprint", "snow_depth")])
```



Running a `cor()` test on continuous variables

cor test on vars except 'region' and 'season'

```
contVar = cor(ungulates[,c("ndvi", "dist_water", "elevation", "human_footprint", "snow_depth")])
contVar
```

```
##           ndvi dist_water elevation human_footprint
## ndvi          1.00000000 -0.08719734 -0.097764589   -0.05957976
## dist_water    -0.08719734  1.00000000  0.084521535   -0.01878825
## elevation     -0.09776459  0.08452154  1.000000000   -0.04244559
## human_footprint -0.05957976 -0.01878825 -0.042445590    1.00000000
## snow_depth    -0.69689343  0.10191401 -0.003084457    0.03084055
##           snow_depth
## ndvi          -0.696893428
## dist_water      0.101914005
## elevation      -0.003084457
## human_footprint 0.030840547
## snow_depth      1.000000000
```

Based on this there aren't any overly concerning collinearity problems between the continuous variables.

Dealing with the categorical variables

Can convert the categorical variables using `factor()` so can think about using them.

Start by looking at the unique values for *region* and *season* from **ungulates**.

```
unique(ungulates$region)
```

```
## [1] "Central" "South"   "North"
```

```
unique(ungulates$season)
```

```
## [1] "Winter" "Summer"
```

For safe keeping, going to create a new dataframe separate from the original. At first going to exclude *region* and *season*.

```
# New dataframe, can manipulate the data more freely
full_ungulates = ungulates[c(-6,-7)]
```

Now, use `factor()` to turn *region* and *season* into more-plottable data. Still aren't continuous in the way the other variables, but now could use them in a `lm()`, for example.

```
full_ungulates$region = factor(ungulates$region,
                               levels = c("Central", "North", "South"),
                               labels = c("Central", "North", "South"))

full_ungulates$season = factor(ungulates$season,
                               levels = c("Summer", "Winter"),
                               labels = c("Summer", "Winter"))

head(full_ungulates)
```

```
##           ndvi dist_water elevation human_footprint snow_depth density region
## 1 0.6481185  1.5915271 1120.2564      86.46314  21.882679 11.42866 Central
## 2 0.8452634  4.5464169 1394.5372      55.03653   3.198445 18.71176 Central
```

```
## 3 0.3159613 0.1095153 442.1242      91.61587 51.858680 13.44218 Central
## 4 0.6437102 0.3767398 456.8150      45.27318 22.284056 11.67938 Central
## 5 0.7229648 3.9307225 929.4815      61.25426 29.994070 10.86497  South
## 6 0.7916754 1.1748960 2091.0933     36.12533 25.243537 18.32689 Central
##   season
## 1 Winter
## 2 Summer
## 3 Summer
## 4 Winter
## 5 Winter
## 6 Summer
```

Taking a quick look at all the variables. Again, think *density* makes the most sense as the response variable. I think *season* needs to be a predictor. Seems like *human_footprint* is something of an outlier against the other subjects, so might be interesting to include in potential models.

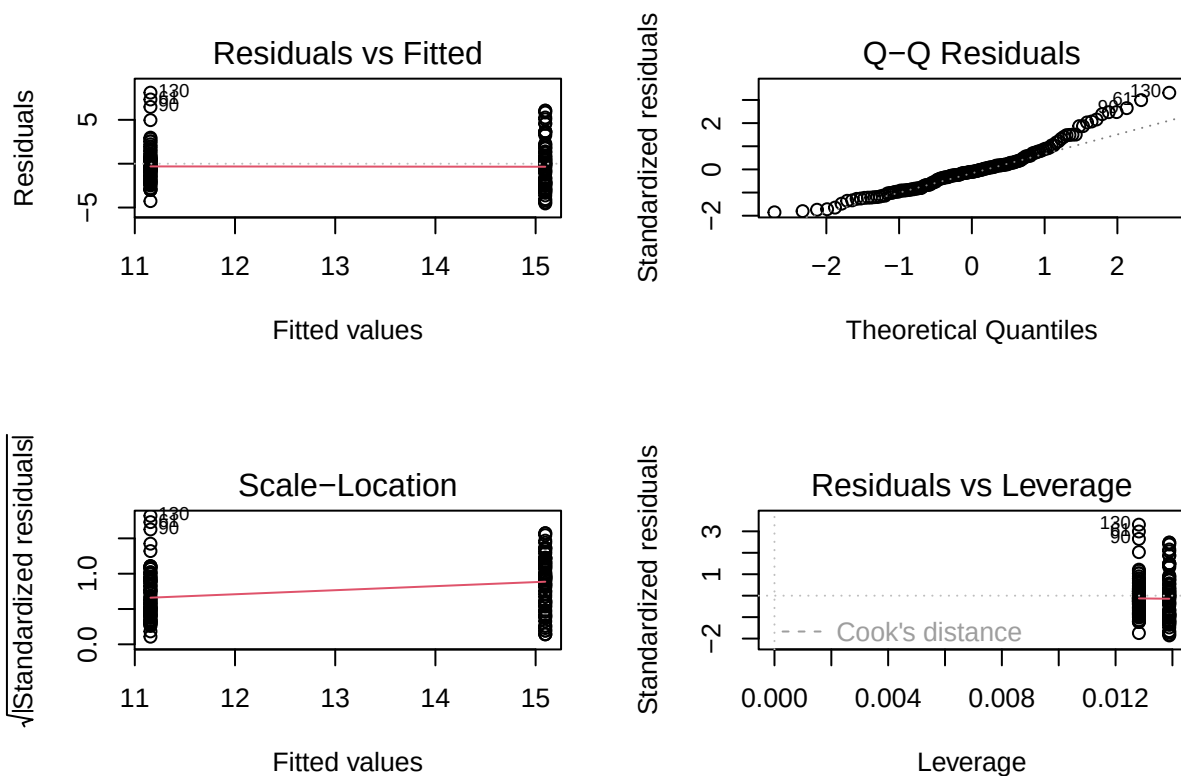
Trying some early lm's

If want to include *season*, can start with preliminary plots to see it against density.

```
# Basic boxplot to see what the initial comparison looks like
#plot(full_ungulates$density ~ full_ungulates$season)
#plot(full_ungulates$season, full_ungulates$density)
```

```
# Starting with the first relationship I'm interested in
base.lm = lm(density ~ season, data = full_ungulates)
```

```
par(mfrow = c(2,2))
plot(base.lm)
```



Yeah, the residuals don't quite work for straight categorical

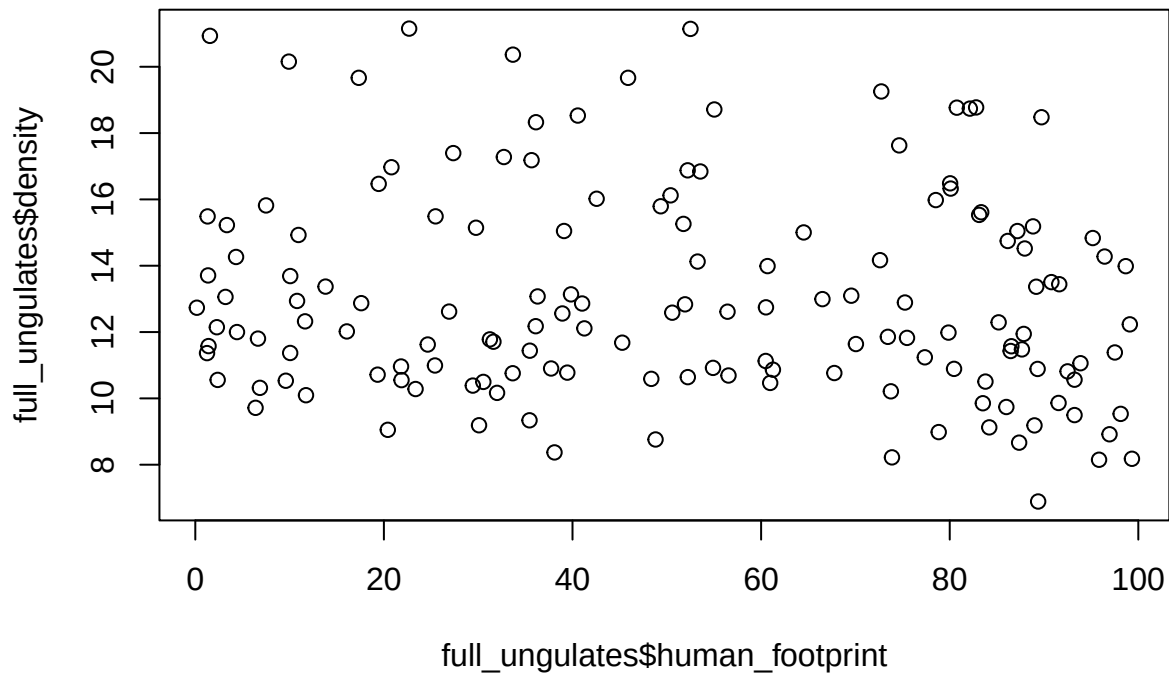
```
summary(base.lm)
```

```
##
## Call:
## lm(formula = density ~ season, data = full_ungulates)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -4.5323 -1.7954 -0.2582  0.9729  8.0987
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   15.0963     0.2901   52.031  <2e-16 ***
## seasonWinter  -3.9400     0.4024  -9.792  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.462 on 148 degrees of freedom
## Multiple R-squared:  0.3932, Adjusted R-squared:  0.3891
## F-statistic: 95.89 on 1 and 148 DF, p-value: < 2.2e-16
```

Get an initial peak at what linear model with *season* might look like. Will run another quick one with *human_footprint*.

```
humanft.lm = lm(density ~ human_footprint, data = full_ungulates)
```

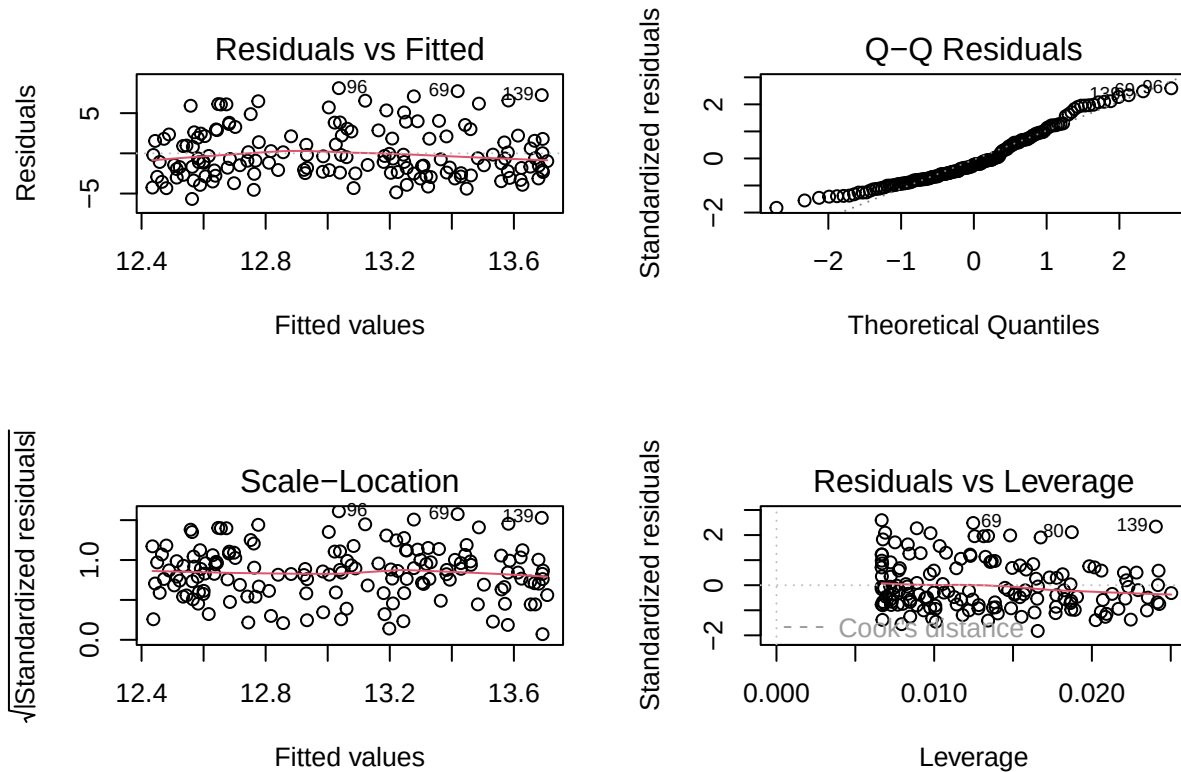
```
plot(full_ungulates$density ~ full_ungulates$human_footprint)
```



```
summary(humanft.lm)
```

```
##
## Call:
## lm(formula = density ~ human_footprint, data = full_ungulates)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -5.6721 -2.3759 -0.7243  2.1183  8.1057
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   13.709673   0.497179  27.575  <2e-16 ***
## human_footprint -0.012831   0.008259  -1.554    0.122
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.135 on 148 degrees of freedom
## Multiple R-squared:  0.01605,    Adjusted R-squared:  0.0094
## F-statistic: 2.414 on 1 and 148 DF,  p-value: 0.1224
```

```
par(mfrow = c(2,2))
plot(humanft.lm)
```



General examination of the variables vs density

Just going to visualize the rest of the variables as well.

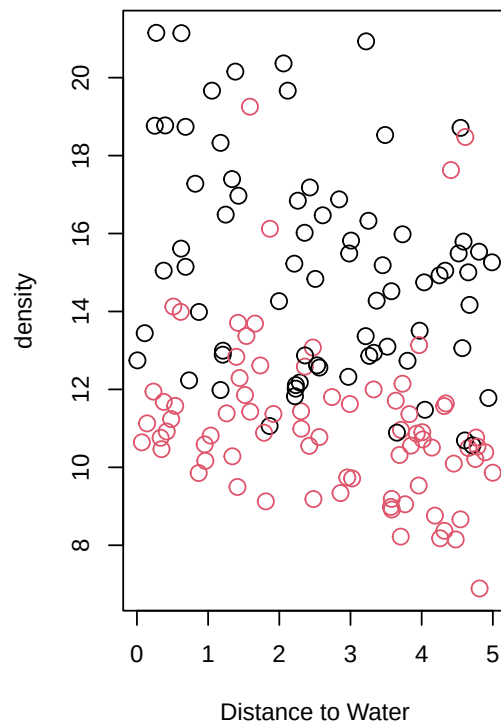
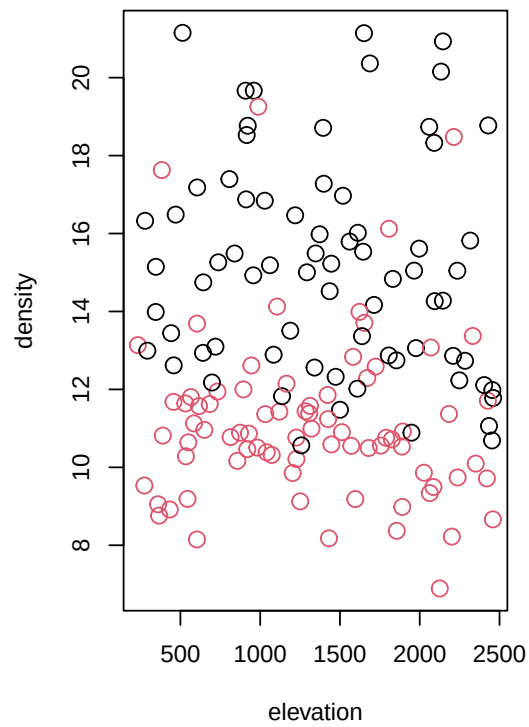
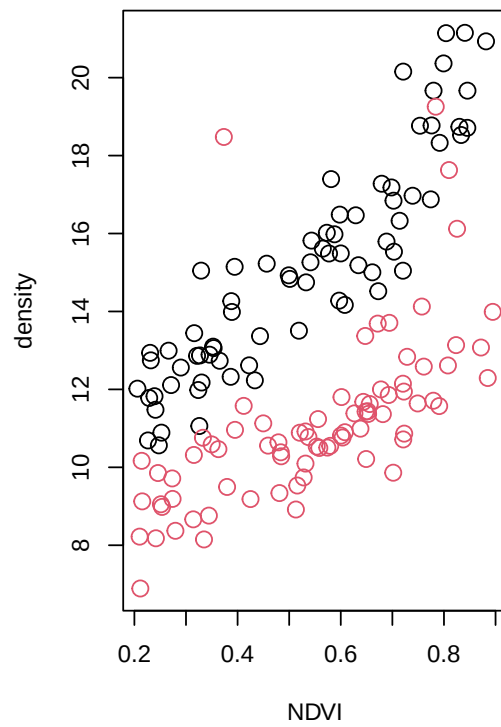
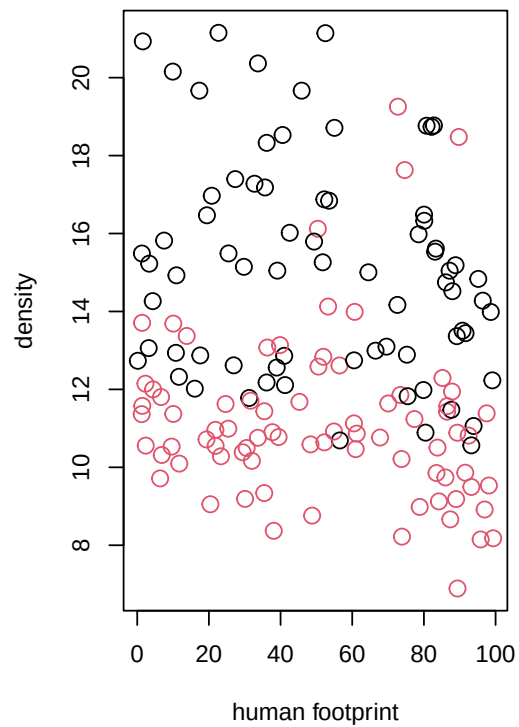
```
# going to plot them all together
par(mfrow = c(2, 2))

# human_footprint
plot(full_ungulates$density ~ full_ungulates$human_footprint,
     xlab = "human footprint", ylab = "density",
     col = full_ungulates$season, cex = 1.5)

# ndvi
plot(full_ungulates$density ~ full_ungulates$ndvi,
     xlab = "NDVI", ylab = "density",
     col = full_ungulates$season, cex = 1.5)

# elevation
plot(full_ungulates$density ~ full_ungulates$elevation,
     xlab = "elevation", ylab = "density",
     col = full_ungulates$season, cex = 1.5)
```

```
# dist_water
plot(full_ungulates$density ~ full_ungulates$dist_water,
     xlab = "Distance to Water", ylab = "density",
     col = full_ungulates$season, cex = 1.5)
```

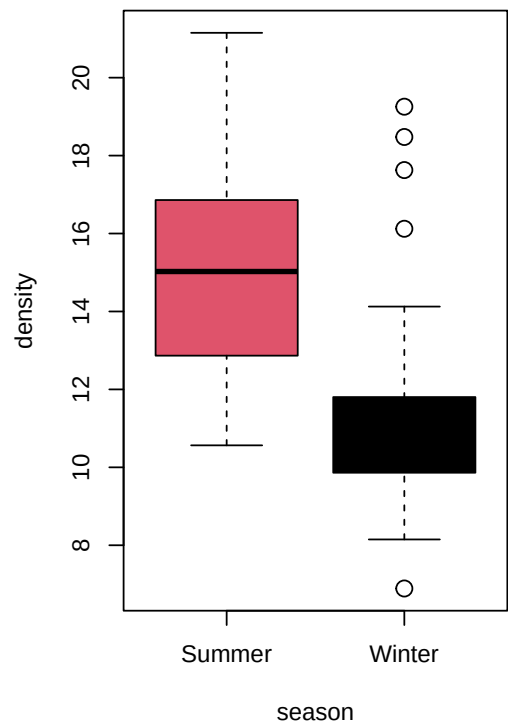
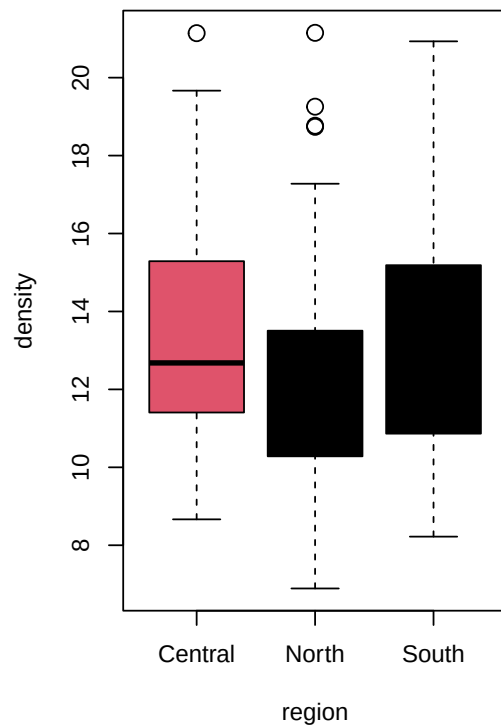
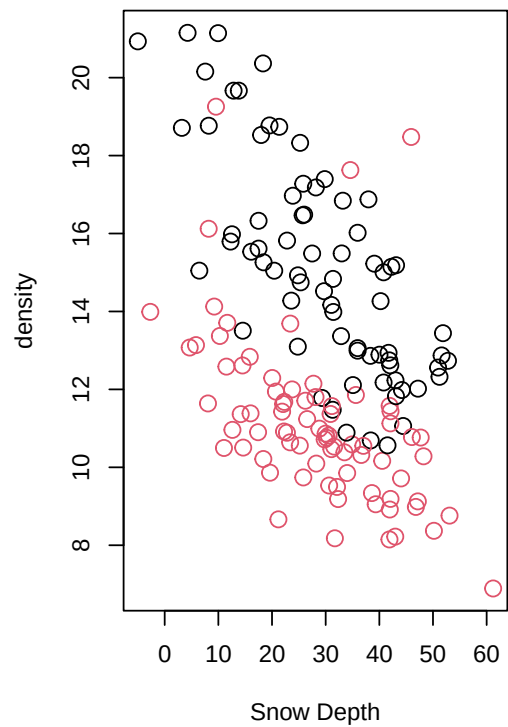


```
par(mfrow = c(2,2))

# elevation
plot(full_ungulates$density ~ full_ungulates$snow_depth,
     xlab = "Snow Depth", ylab = "density",
     col = full_ungulates$season, cex = 1.5)

# region
plot(full_ungulates$density ~ full_ungulates$region,
     xlab = "region", ylab = "density",
     col = full_ungulates$season, cex = 1.5)

# season
plot(full_ungulates$density ~ full_ungulates$season,
     xlab = "season", ylab = "density",
     col = full_ungulates$season, cex = 1.5)
```



Now I need to get a little more serious about how I want to set this up. Need to check how we handle the categorical data, especially for diagnostics. My base idea is just compare the predictors against one another and build complexity, and run the tests we've been learning recently to see what combination of predictors seem best.

TBH that sounds pretty simple, but maybe that's all that's needed.

```
# Just reminding you that hist() and boxplot() and such exist
# hist(full_ungulates$snow_depth)
```

You can make some visualizations, see Lect 17 worksheet. But only with 2 predictors, can't plot well beyond that. Focus on building your model now, adding variables and testing them. Think once you get settled with that, will be able to flesh everything out.

Since *season* and *region* are categorical, should I start the model with a continuous variable?

Setting up my project

I'm spinning my gears a little. First, what is my research question?

Answer: Determining what combination of predictors creates the best model for the density of ungulates.

Ok then.

What have you noticed from your different predictors? > Answer: *season* and *region* are categorical, which means they will feature a bit different on any plot. I need to be sure of how I handle them. If I can't figure it out, stick to the continuous predictors. Otherwise, doesn't seem to be much of a relationship between density and the predictors by themselves. There might be a negative linear relationship between snow depth, that's about it. I don't think will need any polynomial regression stuff here.

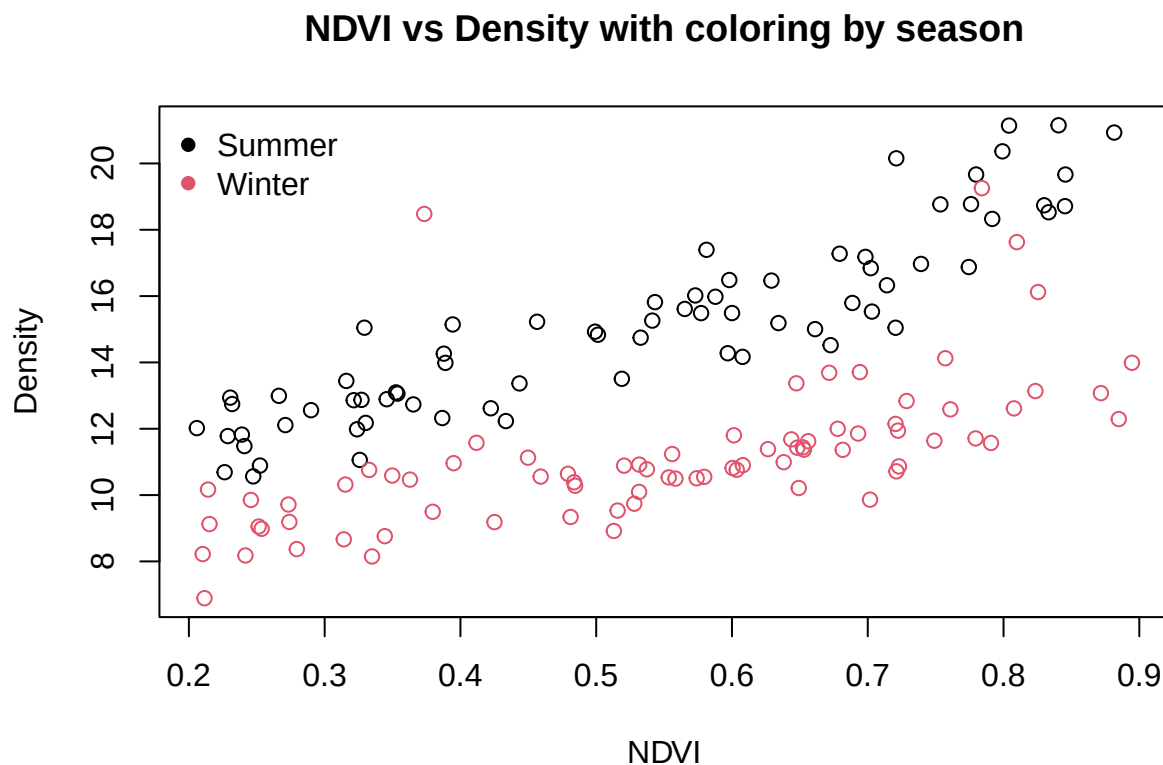
Since I need some structure, I'm going to mimic the 2nd part of Assignment 7. I already did a `pairs()` and `cor()` check on the continuous variables, I'll also do a VIF check. The only one that seemed a little close was NDVI to Snow_Depth.

```
# Checking VIF, one more test of collinearity on cont. vars
full_cont_lm = lm(density ~ ndvi + dist_water + elevation + human_footprint + snow_depth,
                  data = full_ungulates)
vif_vals = vif(full_cont_lm)
VIF = as.data.frame(vif_vals)
VIF
```

```
##           vif_vals
## ndvi           1.990099
## dist_water      1.018396
## elevation       1.029298
## human_footprint 1.006707
## snow_depth      1.973003
```

Don't see anything too bad, everything is $VIF < 5$, so no continuous variables should need removing.

```
# Plot of ndvi vs density, but color coded with season
plot(density ~ ndvi, data = full_ungulates,
     col = full_ungulates$season,
     xlab = "NDVI", ylab = "Density",
     main = "NDVI vs Density with coloring by season")
legend("topleft", legend = levels(full_ungulates$season), pch = 16, col = 1:3, bty = "n")
```



Ok so I was having reservations about running *season* and *region*, but after the plot above, it clearly changes how the data looks. I don't think the data is going to make sense without at least one of season or region. So, let's see how I want to build the model.

My two ideas are either start with all continuous variables and add *season* and or *region*, or start with a categorical var and add on the continuous.

But yeah suddenly I feel like I've got something to work with. TYL

Model diagnostics

Honestly right now, I'm not sure what I should be running.

Mucking around models

I did some random tests to try and get my eye in. Going to put them in a code scrap-pile here:

1. For the full predictor model, minus the categoricals, `-> full_cont_lm = lm(density ~ ndvi + dist_water + elevation + human_footprint + snow_depth, data = full_ungulates)`

2. Same thing but using , -> `asteric_lm = lm(density ~ ndvi+dist_water+elevation+human_footprint*snow_depth, data = full_ungulates)`
3. All variables except region, -> `all_min1_lm = lm(density ~ ndvi+dist_water+elevation+human_footprint+snow_depth, data = full_ungulates)`
4. Checking if the above was the same without using `as.factor()` on season, -> `lm1 = lm(density ~ ndvi+dist_water+elevation+human_footprint+snow_depth+season, data = full_ungulates)`
5. basically `full_cont_lm` with , -> `lm2 = lm(density ~ season+ndvi+dist_water+elevation+human_footprint+snow_depth, data = full_ungulates)`
6. Checking interaction effect `lm3 = lm(density ~ season:ndvi, data = full_ungulates)`
7. Checking `lm4 = lm(density ~ season + ndvi, data = full_ungulates)`
8. Checking just plain season `lm5 = lm(density ~ season, data = full_ungulates)`

Building some models

So having spent some time with that, where do we go?

Can build a non-model I suppose.

I think I want to start with seasons, and then add on other continuous variables. I don't know if I want to add in regions right now. But I think seasons at least is needed to get the linear model going.

```
# Going to build out my models
# I'm going to use '+' for now, not '*' or ':'

# non-model
model_0 = lm(density ~ 1, data = full_ungulates)

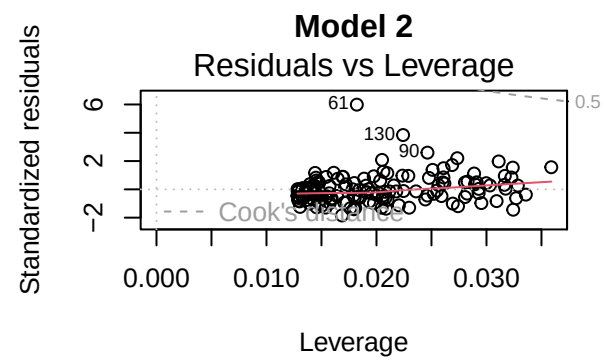
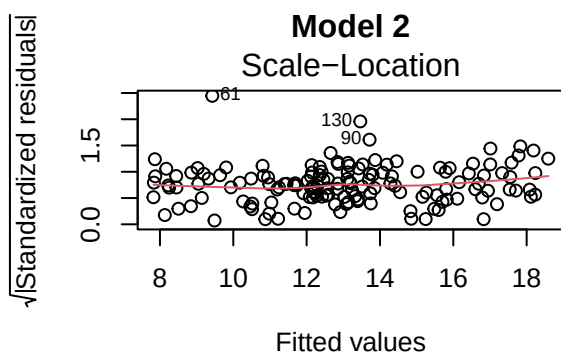
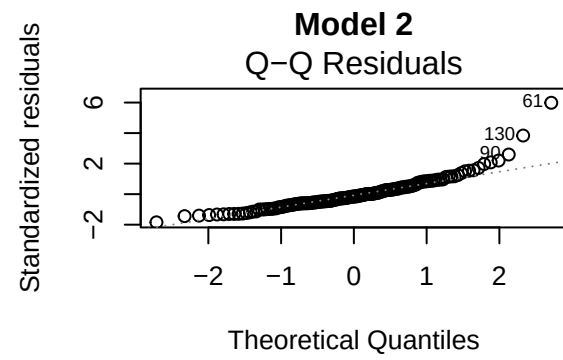
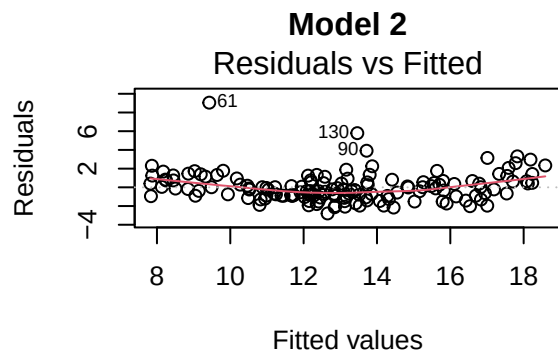
model_1 = lm(density ~ season, data = full_ungulates)
model_2 = lm(density ~ season + ndvi, data = full_ungulates)
model_3 = lm(density ~ season + ndvi + dist_water, data = full_ungulates)
model_4 = lm(density ~ season + ndvi + dist_water + elevation, data = full_ungulates)
model_5 = lm(density ~ season + ndvi + dist_water + elevation + human_footprint,
             data = full_ungulates)
model_6 = lm(density ~ season + ndvi + dist_water + elevation + human_footprint
             + snow_depth, data = full_ungulates)
```

I'm going to stop there for a moment. I think `dredge()` can take all the predictors later. The caveat with `dredge()` is use it for model comparison, having crafted a model. You don't get any of the ecological implications if you just shove in all the predictors.

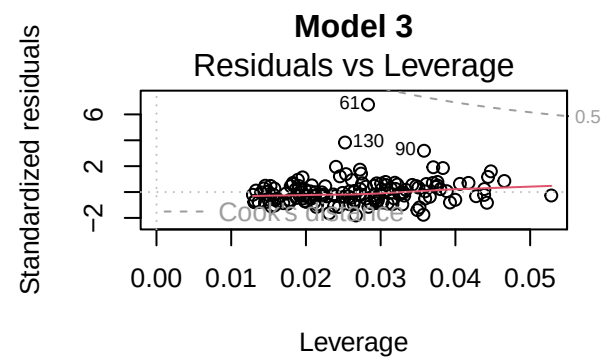
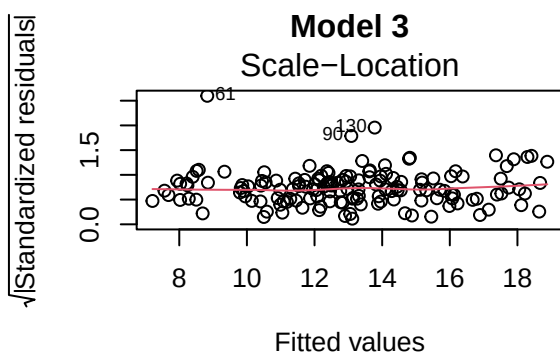
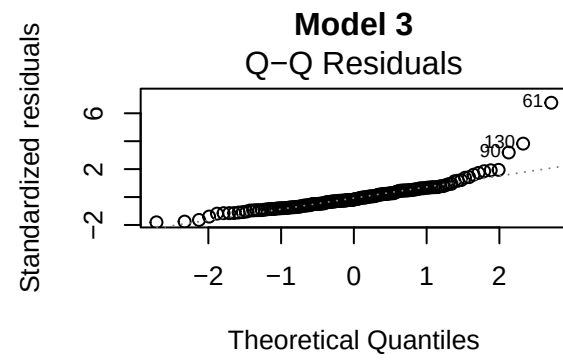
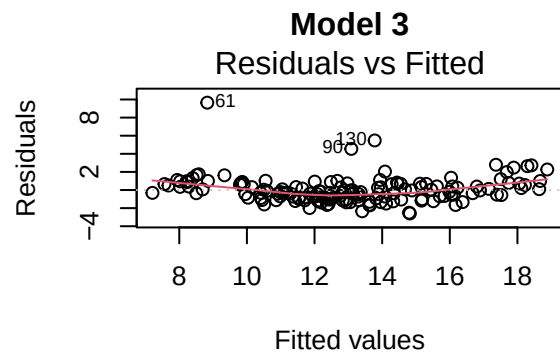
I think season, ndvi, dist_water might be interesting variables. I don't know if elevation is useful, so lets find out.

```
# Before I check summary(), going to check diagnostics. Already saw model_1 earlier,
# and model_0 is worthless

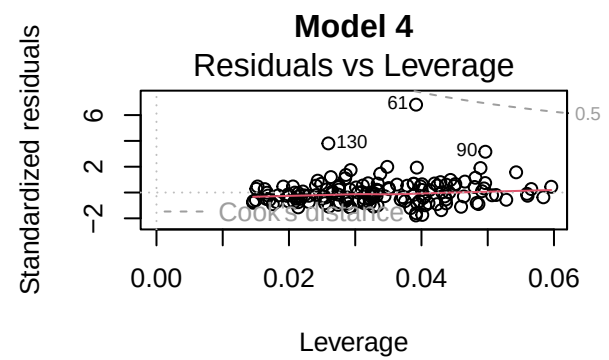
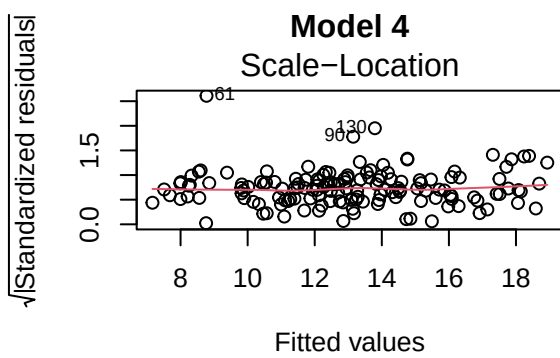
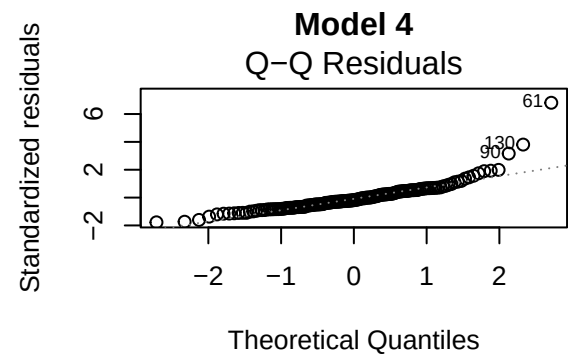
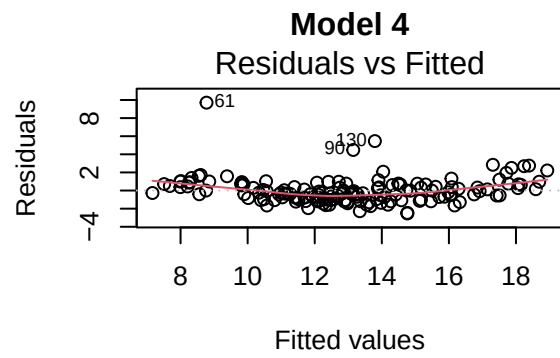
par(mfrow = c(2,2))
plot(model_2, main = "Model 2")
```



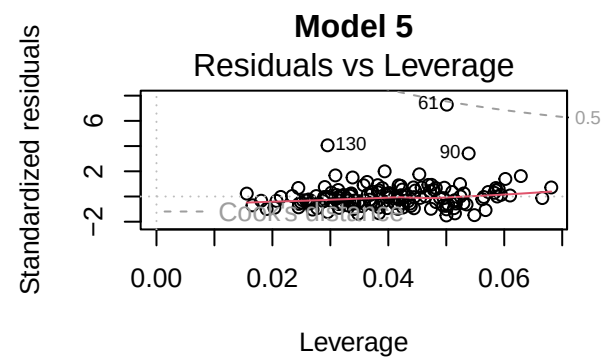
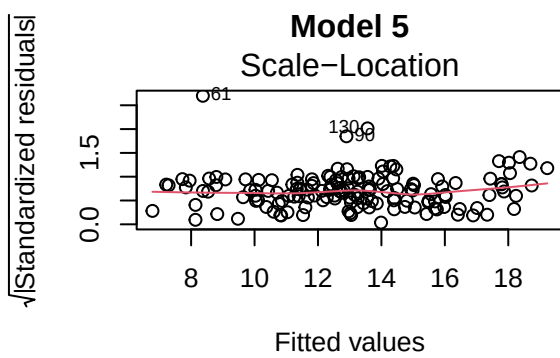
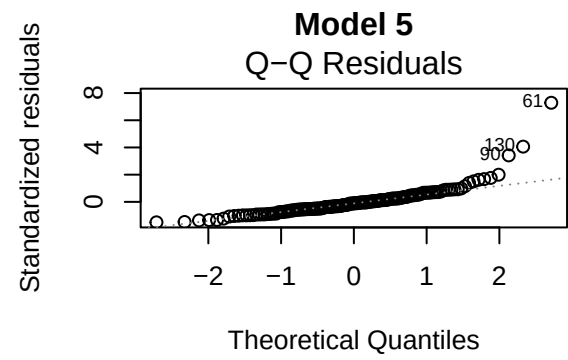
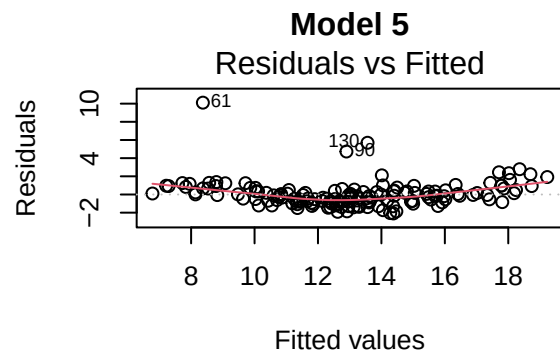
```
plot(model_3, main = "Model 3")
```



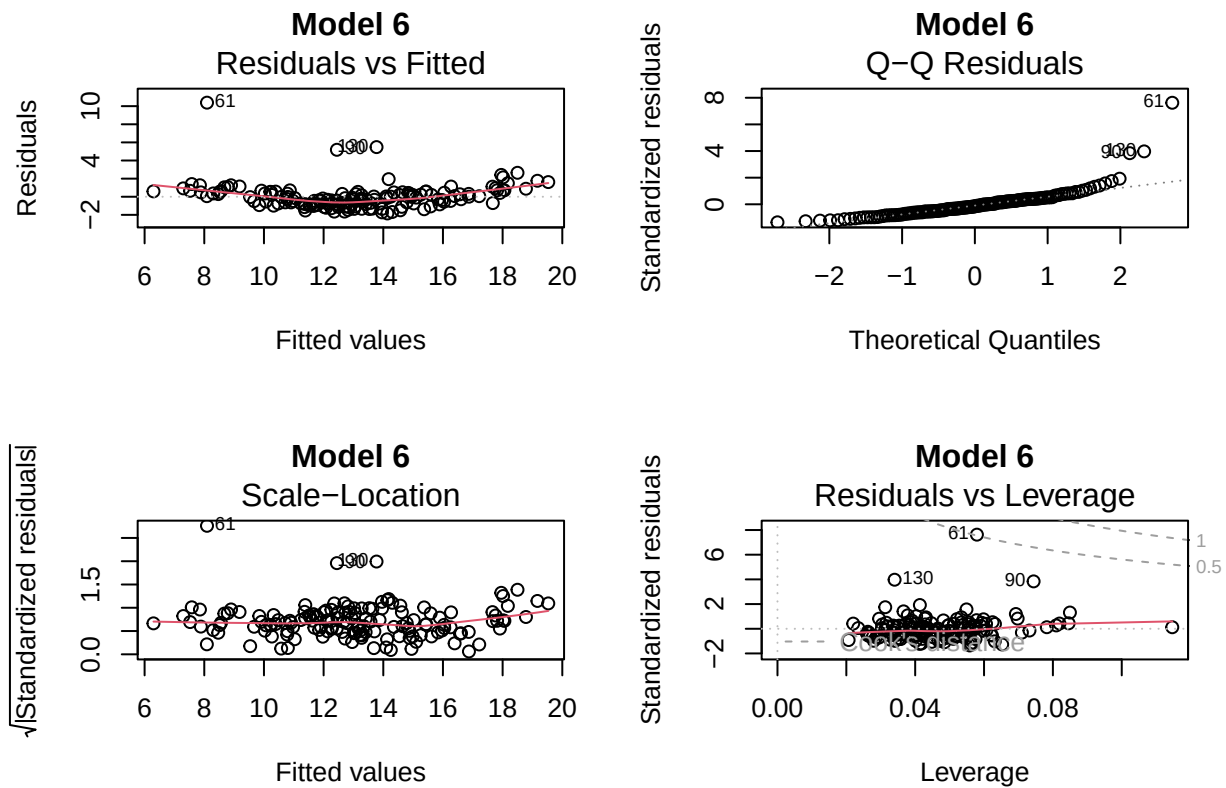
```
plot(model_4, main = "Model 4")
```



```
plot(model_5, main = "Model 5")
```



```
plot(model_6, main = "Model 6")
```



Possibly some normality issues especially higher up. But the others look alright. Can do some shapiro.wilks tests

NEVERMIND, shapiro.test() is for the underlying data, not the model

```
shapiro.test(full_ungulates$ndvi)
```

```
##
##  Shapiro-Wilk normality test
##
## data:  full_ungulates$ndvi
## W = 0.95206, p-value = 4.874e-05
```

```
shapiro.test(full_ungulates$dist_water)
```

```
##
##  Shapiro-Wilk normality test
##
## data:  full_ungulates$dist_water
## W = 0.94491, p-value = 1.266e-05
```

```
shapiro.test(full_ungulates$elevation)
```

```
##
##  Shapiro-Wilk normality test
```

```
##
## data: full_ungulates$elevation
## W = 0.96012, p-value = 0.0002513
```

```
shapiro.test(full_ungulates$human_footprint)
```

```
##
## Shapiro-Wilk normality test
##
## data: full_ungulates$human_footprint
## W = 0.93003, p-value = 9.98e-07
```

```
shapiro.test(full_ungulates$snow_depth)
```

```
##
## Shapiro-Wilk normality test
##
## data: full_ungulates$snow_depth
## W = 0.98971, p-value = 0.3415
```

So just doing a quick check on all the continuous variables, snow_depth was the odd man out.

```
summary(model_0)
```

```
##
## Call:
## lm(formula = density ~ 1, data = full_ungulates)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -6.1568 -2.2877 -0.7851  2.0733  8.1028
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  13.0475     0.2572   50.73  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.15 on 149 degrees of freedom
```

```
summary(model_1)
```

```
##
## Call:
## lm(formula = density ~ season, data = full_ungulates)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -4.5323 -1.7954 -0.2582  0.9729  8.0987
##
## Coefficients:
```

```
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)   15.0963     0.2901  52.031  <2e-16 ***
## seasonWinter  -3.9400     0.4024  -9.792  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.462 on 148 degrees of freedom
## Multiple R-squared:  0.3932, Adjusted R-squared:  0.3891
## F-statistic: 95.89 on 1 and 148 DF,  p-value: < 2.2e-16
```

```
summary(model_2)
```

```
##
## Call:
## lm(formula = density ~ season + ndvi, data = full_ungulates)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.7917 -0.9019 -0.2303  0.6702  9.0507
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)    9.9258     0.3803   26.10  <2e-16 ***
## seasonWinter   -4.1687     0.2499  -16.68  <2e-16 ***
## ndvi            9.8287     0.6369   15.43  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.526 on 147 degrees of freedom
## Multiple R-squared:  0.7684, Adjusted R-squared:  0.7652
## F-statistic: 243.8 on 2 and 147 DF,  p-value: < 2.2e-16
```

```
summary(model_3)
```

```
##
## Call:
## lm(formula = density ~ season + ndvi + dist_water, data = full_ungulates)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.5875 -0.8734 -0.2517  0.6568  9.6492
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  10.90028     0.43216  25.223  < 2e-16 ***
## seasonWinter -4.12353     0.23762 -17.354  < 2e-16 ***
## ndvi          9.60443     0.60751  15.809  < 2e-16 ***
## dist_water   -0.33264     0.08096  -4.109 6.61e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.45 on 146 degrees of freedom
## Multiple R-squared:  0.7924, Adjusted R-squared:  0.7881
## F-statistic: 185.8 on 3 and 146 DF,  p-value: < 2.2e-16
```

```
summary(model_4)
```

```
##
## Call:
## lm(formula = density ~ season + ndvi + dist_water + elevation,
##     data = full_ungulates)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.5296 -0.9142 -0.2559  0.6600  9.7033
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  1.100e+01  5.167e-01  21.291 < 2e-16 ***
## seasonWinter -4.132e+00  2.396e-01 -17.246 < 2e-16 ***
## ndvi         9.586e+00  6.115e-01  15.675 < 2e-16 ***
## dist_water   -3.303e-01  8.148e-02  -4.054 8.2e-05 ***
## elevation    -6.756e-05  1.895e-04  -0.357  0.722
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.454 on 145 degrees of freedom
## Multiple R-squared:  0.7926, Adjusted R-squared:  0.7869
## F-statistic: 138.5 on 4 and 145 DF, p-value: < 2.2e-16
```

```
summary(model_5)
```

```
##
## Call:
## lm(formula = density ~ season + ndvi + dist_water + elevation +
##     human_footprint, data = full_ungulates)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.0849 -0.7599 -0.1499  0.4532 10.1061
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  1.163e+01  5.566e-01  20.896 < 2e-16 ***
## seasonWinter -4.138e+00  2.345e-01 -17.642 < 2e-16 ***
## ndvi         9.480e+00  5.999e-01  15.804 < 2e-16 ***
## dist_water   -3.346e-01  7.977e-02  -4.195 4.76e-05 ***
## elevation    -9.147e-05  1.857e-04  -0.493  0.62298
## human_footprint -1.019e-02  3.762e-03  -2.710  0.00754 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.423 on 144 degrees of freedom
## Multiple R-squared:  0.8026, Adjusted R-squared:  0.7958
## F-statistic: 117.1 on 5 and 144 DF, p-value: < 2.2e-16
```

```
summary(model_6)
```

```
##
## Call:
## lm(formula = density ~ season + ndvi + dist_water + elevation +
##     human_footprint + snow_depth, data = full_ungulates)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.8440 -0.7421 -0.2100  0.4847 10.3856
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   13.1590593  0.8737641  15.060 < 2e-16 ***
## seasonWinter  -4.1405932  0.2313152 -17.900 < 2e-16 ***
## ndvi           8.1857860  0.8254572   9.917 < 2e-16 ***
## dist_water    -0.3229581  0.0788345  -4.097 6.99e-05 ***
## elevation     -0.0001351  0.0001841  -0.734  0.4643
## human_footprint -0.0103501  0.0037105  -2.789  0.0060 **
## snow_depth    -0.0275450  0.0122495  -2.249  0.0261 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.404 on 143 degrees of freedom
## Multiple R-squared:  0.8094, Adjusted R-squared:  0.8014
## F-statistic: 101.2 on 6 and 143 DF,  p-value: < 2.2e-16
```

Interpreting the models So in each of these models, only seasonWinter's estimate is relative to seasonSummer, aka Intercept. Everything else is assuming all other variables are held constant.

I was getting confused, so ran the following code.

```
#additive.lm = lm(Sepal.Length ~ as.factor(Species) + Sepal.Width, data = iris)
#summary(additive.lm)
```

This is essentially what I've done. I've a "Diff Intercept, Same Slope" setup.

But what I think I need, based on ecological logic, is "Different Slopes, Different Intercepts".

```
all_diff_lm = lm(Sepal.Length ~ as.factor(Species) * Sepal.Width, data = iris)
summary(all_diff_lm)
```

```
##
## Call:
## lm(formula = Sepal.Length ~ as.factor(Species) * Sepal.Width,
##     data = iris)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.26067 -0.25861 -0.03305  0.18929  1.44917
##
## Coefficients:
```

```
##                                Estimate Std. Error t value Pr(>|t|)
## (Intercept)                    2.6390     0.5715   4.618 8.53e-06
## as.factor(Species)versicolor    0.9007     0.7988   1.128  0.261
## as.factor(Species)virginica     1.2678     0.8162   1.553  0.123
## Sepal.Width                    0.6905     0.1657   4.166 5.31e-05
## as.factor(Species)versicolor:Sepal.Width 0.1746     0.2599   0.672  0.503
## as.factor(Species)virginica:Sepal.Width 0.2110     0.2558   0.825  0.411
##
## (Intercept)                    ***
## as.factor(Species)versicolor
## as.factor(Species)virginica
## Sepal.Width                    ***
## as.factor(Species)versicolor:Sepal.Width
## as.factor(Species)virginica:Sepal.Width
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.4397 on 144 degrees of freedom
## Multiple R-squared:  0.7274, Adjusted R-squared:  0.718
## F-statistic: 76.87 on 5 and 144 DF, p-value: < 2.2e-16
```

TBH, I'm starting to think its easier if I just leave season as it was originally and just use as.factor(). But we'll stick with it for now.

Above code chunk shows what “Different Slopes, Different Intercepts” looks like with one variable. It gets complicated the more you start adding.

```
new_lm = lm(density ~ season * (ndvi + dist_water), data = full_ungulates)
summary(new_lm)
```

```
##
## Call:
## lm(formula = density ~ season * (ndvi + dist_water), data = full_ungulates)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.5655 -0.7165 -0.2109  0.4163  9.0369
##
## Coefficients:
##                                Estimate Std. Error t value Pr(>|t|)
## (Intercept)                    9.8990     0.5498  18.005 < 2e-16 ***
## seasonWinter                   -1.9341     0.7935  -2.437 0.016018 *
## ndvi                          12.0515     0.7944  15.171 < 2e-16 ***
## dist_water                     -0.4437     0.1121  -3.960 0.000118 ***
## seasonWinter:ndvi              -5.0188     1.1392  -4.406 2.05e-05 ***
## seasonWinter:dist_water        0.1944     0.1523   1.276 0.203854
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.359 on 144 degrees of freedom
## Multiple R-squared:  0.8201, Adjusted R-squared:  0.8139
## F-statistic: 131.3 on 5 and 144 DF, p-value: < 2.2e-16
```

But I think I'm finally understanding what this says. Intercept and seasonWinter are your your starting intercepts. Aka, density in summer starts at this, in winter starts at that.

Then, you have your summer slopes for density. In the summer, ndvi causes this change, dist_water causes this change.

THEN, you have your winter slopes for density. In winter, ndvi causes this change, dist_water causes this change.

Building some ecologically logical models

Now that I understand what's going on with the models, I'm going to remake those models, but using density ~ season * (all the other predictors)

At great risk I'm going to do model.# instead of model_#

```
# Making Diff Int, Diff Slope models. These might be 'the ones'
```

```
model.1 = lm(density ~ season, data = full_ungulates)
model.2 = lm(density ~ season * ndvi, data = full_ungulates)
model.3 = lm(density ~ season * (ndvi + dist_water), data = full_ungulates)
model.4 = lm(density ~ season * (ndvi + dist_water + elevation), data = full_ungulates)
model.5 = lm(density ~ season * (ndvi + dist_water + elevation + human_footprint),
             data = full_ungulates)
model.6 = lm(density ~ season * (ndvi + dist_water + elevation + human_footprint
                               + snow_depth), data = full_ungulates)
```

```
summary(model.1)
```

```
##
## Call:
## lm(formula = density ~ season, data = full_ungulates)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -4.5323 -1.7954 -0.2582  0.9729  8.0987
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   15.0963     0.2901  52.031  <2e-16 ***
## seasonWinter  -3.9400     0.4024  -9.792  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.462 on 148 degrees of freedom
## Multiple R-squared:  0.3932, Adjusted R-squared:  0.3891
## F-statistic: 95.89 on 1 and 148 DF, p-value: < 2.2e-16
```

```
summary(model.2)
```

```
##
## Call:
## lm(formula = density ~ season * ndvi, data = full_ungulates)
```

```
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.4412 -0.8024 -0.1107  0.5441  8.5939
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      8.6279     0.4752  18.154 < 2e-16 ***
## seasonWinter     -1.4450     0.6917  -2.089  0.0384 *
## ndvi             12.2959     0.8433  14.581 < 2e-16 ***
## seasonWinter:ndvi -5.0627     1.2080  -4.191 4.79e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.447 on 146 degrees of freedom
## Multiple R-squared:  0.7933, Adjusted R-squared:  0.789
## F-statistic: 186.7 on 3 and 146 DF, p-value: < 2.2e-16
```

```
summary(model.3)
```

```
##
## Call:
## lm(formula = density ~ season * (ndvi + dist_water), data = full_ungulates)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.5655 -0.7165 -0.2109  0.4163  9.0369
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      9.8990     0.5498  18.005 < 2e-16 ***
## seasonWinter     -1.9341     0.7935  -2.437 0.016018 *
## ndvi             12.0515     0.7944  15.171 < 2e-16 ***
## dist_water       -0.4437     0.1121  -3.960 0.000118 ***
## seasonWinter:ndvi -5.0188     1.1392  -4.406 2.05e-05 ***
## seasonWinter:dist_water 0.1944     0.1523   1.276 0.203854
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.359 on 144 degrees of freedom
## Multiple R-squared:  0.8201, Adjusted R-squared:  0.8139
## F-statistic: 131.3 on 5 and 144 DF, p-value: < 2.2e-16
```

```
summary(model.4)
```

```
##
## Call:
## lm(formula = density ~ season * (ndvi + dist_water + elevation),
##     data = full_ungulates)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.4348 -0.7308 -0.1994  0.3929  9.0170
```

```
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    10.1279589   0.6710231   15.093 < 2e-16 ***
## seasonWinter    -2.1961433   0.9451576   -2.324  0.02157 *
## ndvi            12.0064307   0.8024075   14.963 < 2e-16 ***
## dist_water      -0.4384171   0.1130406   -3.878  0.00016 ***
## elevation       -0.0001530   0.0002541   -0.602  0.54796
## seasonWinter:ndvi -4.9674676   1.1498530   -4.320  2.91e-05 ***
## seasonWinter:dist_water  0.1882186   0.1536759    1.225  0.22269
## seasonWinter:elevation  0.0001777   0.0003562    0.499  0.61861
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.367 on 142 degrees of freedom
## Multiple R-squared:  0.8206, Adjusted R-squared:  0.8118
## F-statistic: 92.8 on 7 and 142 DF, p-value: < 2.2e-16
```

```
summary(model.5)
```

```
##
## Call:
## lm(formula = density ~ season * (ndvi + dist_water + elevation +
##     human_footprint), data = full_ungulates)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.8727 -0.6562 -0.1554  0.3670  9.1787
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    11.1540510   0.7104676   15.700 < 2e-16 ***
## seasonWinter    -2.9597574   1.0065678   -2.940  0.00384 **
## ndvi            11.8878091   0.7740796   15.357 < 2e-16 ***
## dist_water      -0.4478475   0.1089781   -4.110  6.72e-05 ***
## elevation       -0.0001727   0.0002449   -0.705  0.48192
## human_footprint -0.0175647   0.0050355   -3.488  0.00065 ***
## seasonWinter:ndvi -4.9060668   1.1107844   -4.417  1.99e-05 ***
## seasonWinter:dist_water  0.1962828   0.1481410    1.325  0.18734
## seasonWinter:elevation  0.0001831   0.0003437    0.533  0.59511
## seasonWinter:human_footprint  0.0134990   0.0069716    1.936  0.05485 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.317 on 140 degrees of freedom
## Multiple R-squared:  0.8357, Adjusted R-squared:  0.8252
## F-statistic: 79.14 on 9 and 140 DF, p-value: < 2.2e-16
```

```
summary(model.6)
```

```
##
## Call:
## lm(formula = density ~ season * (ndvi + dist_water + elevation +
```

```
##      human_footprint + snow_depth), data = full_ungulates)
##
## Residuals:
##      Min        1Q    Median        3Q        Max
## -1.6534 -0.6197 -0.1840  0.3412  9.2775
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      14.1357657   1.1543874   12.245 < 2e-16 ***
## seasonWinter      -5.3442646   1.5853837   -3.371 0.000972 ***
## ndvi              9.4551075   1.0651626    8.877 3.24e-15 ***
## dist_water       -0.4669509   0.1058479   -4.412 2.05e-05 ***
## elevation        -0.0002338   0.0002383   -0.981 0.328145
## human_footprint  -0.0183673   0.0048895   -3.756 0.000253 ***
## snow_depth       -0.0519981   0.0161527   -3.219 0.001603 **
## seasonWinter:ndvi  -2.9967403   1.5070164   -1.989 0.048732 *
## seasonWinter:dist_water  0.2274918   0.1447828    1.571 0.118412
## seasonWinter:elevation  0.0002224   0.0003353    0.663 0.508222
## seasonWinter:human_footprint 0.0143307   0.0067654    2.118 0.035947 *
## seasonWinter:snow_depth  0.0408788   0.0224614    1.820 0.070933 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.277 on 138 degrees of freedom
## Multiple R-squared:  0.8477, Adjusted R-squared:  0.8356
## F-statistic: 69.84 on 11 and 138 DF,  p-value: < 2.2e-16
```

Now, about 3 hours later, can interpret our results. First two rows of estimates are the intercepts, the starting densities (when all variables = 0) in summer and winter. Then, how density changes in the summer for each var, when all other vars = 0 Then, how density changes in the winter for each var, when all other vars = 0

We see that consistently, there is difference between winter and summer. Consistently, we see NDVI, dist_water, human_footprint as significant for summer We see elevation is insignificant in summer

In winter, only NDVI and human_footprint seem to see a significantly change of their effect on density. However, it's noteworthy that only when snow_depth was added did human_footprint become significant in the winter, and NDVI reduced in magnitude and became less significant than it had been.

We see a reduction in R^2 when elevation was added, and it was never a significant predictor in either summer or winter. I suspect elevation could be more relevant if we considered region, but right now I'm not.

I want to see the model without elevation

```
# model 6, except without elevation
model.6_noElev = lm(density ~ season * (ndvi + dist_water + human_footprint
      + snow_depth), data = full_ungulates)
summary(model.6_noElev)
```

```
##
## Call:
## lm(formula = density ~ season * (ndvi + dist_water + human_footprint +
##      snow_depth), data = full_ungulates)
##
## Residuals:
```

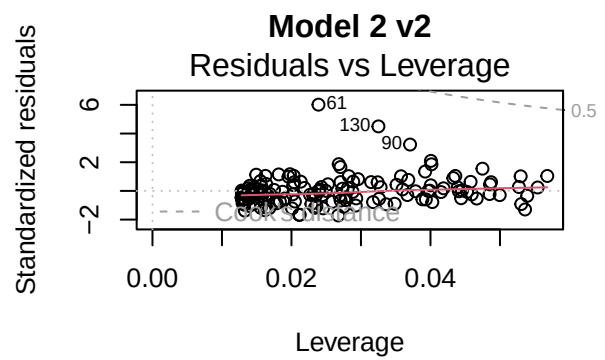
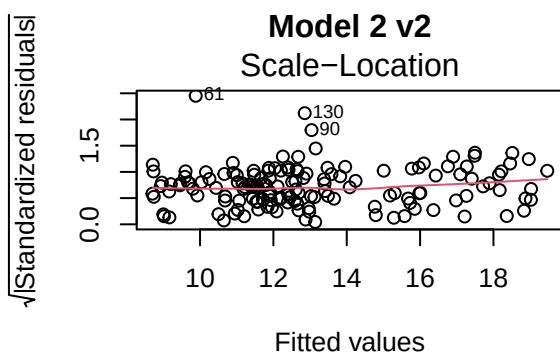
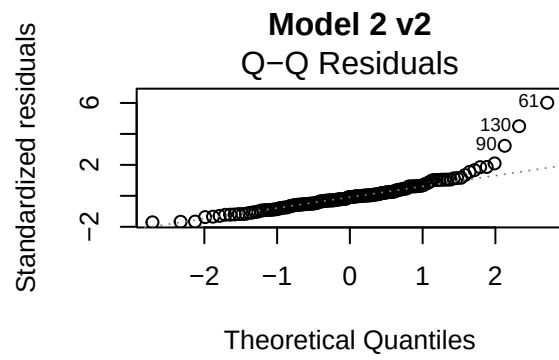
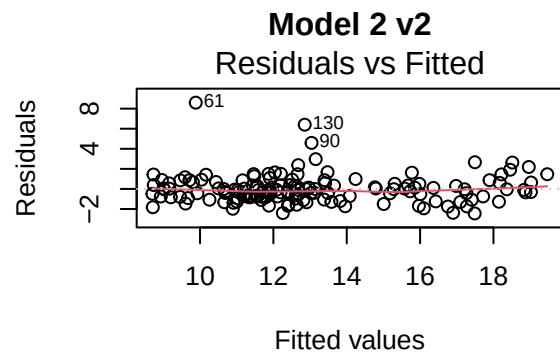
```
##      Min      1Q  Median      3Q      Max
## -1.6615 -0.6356 -0.1249  0.3358  9.2670
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    13.709406   1.065549  12.866 < 2e-16 ***
## seasonWinter    -4.939175   1.454695  -3.395 0.000892 ***
## ndvi             9.583372   1.053200   9.099 8.19e-16 ***
## dist_water     -0.474469   0.105180  -4.511 1.35e-05 ***
## human_footprint -0.018238   0.004870  -3.745 0.000263 ***
## snow_depth     -0.050735   0.016042  -3.163 0.001917 **
## seasonWinter:ndvi -3.117343   1.487458  -2.096 0.037904 *
## seasonWinter:dist_water  0.234509   0.143677   1.632 0.104884
## seasonWinter:human_footprint 0.014217   0.006731   2.112 0.036465 *
## seasonWinter:snow_depth  0.039714   0.022250   1.785 0.076446 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.272 on 140 degrees of freedom
## Multiple R-squared:  0.8467, Adjusted R-squared:  0.8368
## F-statistic: 85.89 on 9 and 140 DF,  p-value: < 2.2e-16
```

We see a slight increase in R^2 , but nothing really changes.

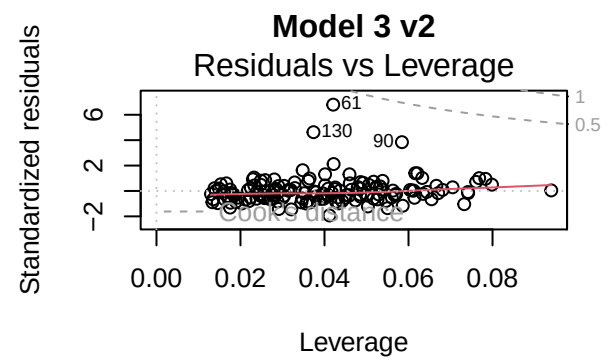
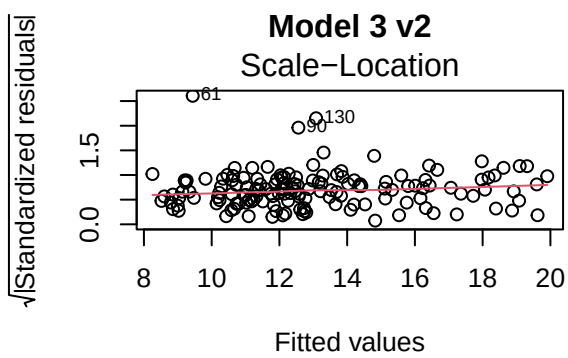
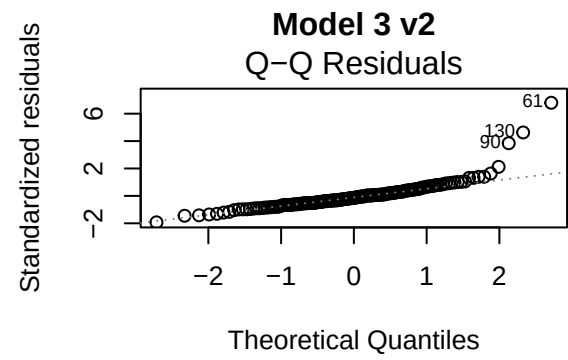
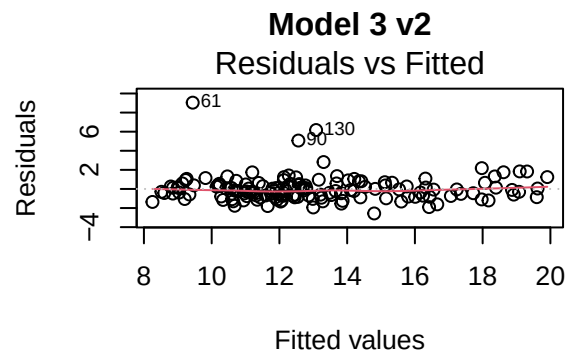
Running Some Diagnostics (and noticing trouble)

I'm going to run diagnostics on these models, which are different than the ones I did earlier.

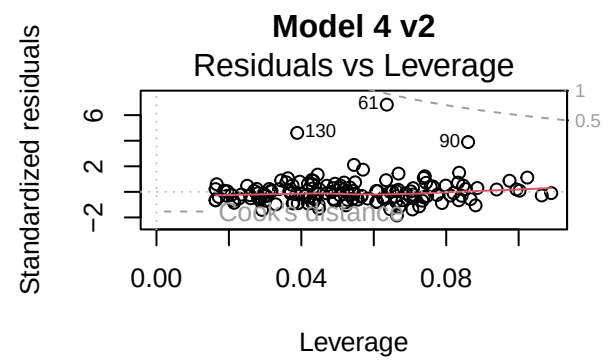
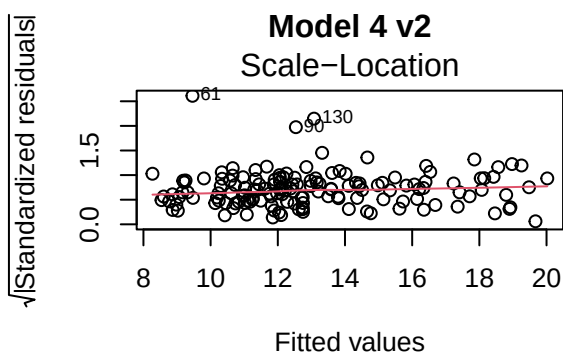
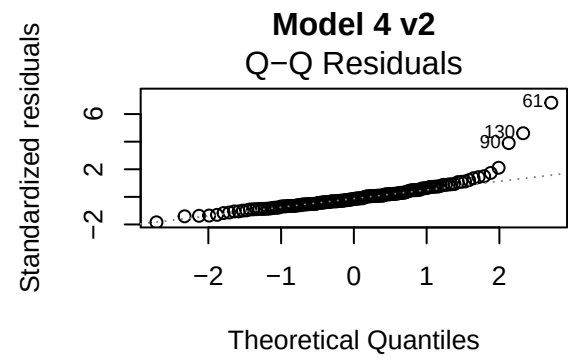
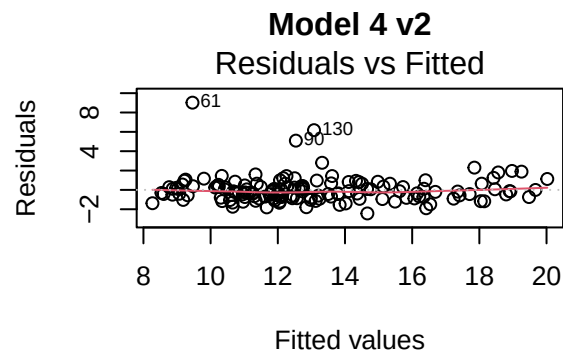
```
# Diagnostics on updated models
par(mfrow = c(2,2))
plot(model.2, main = "Model 2 v2")
```



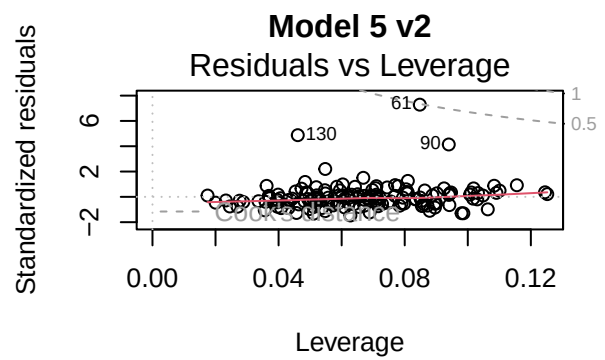
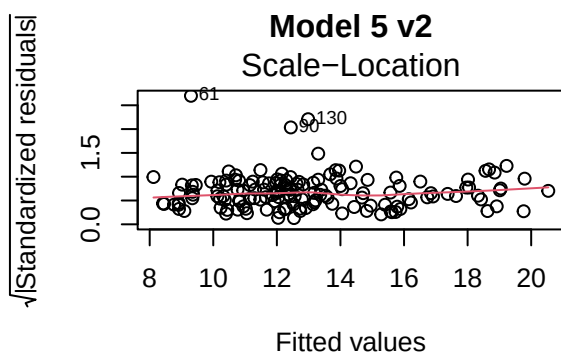
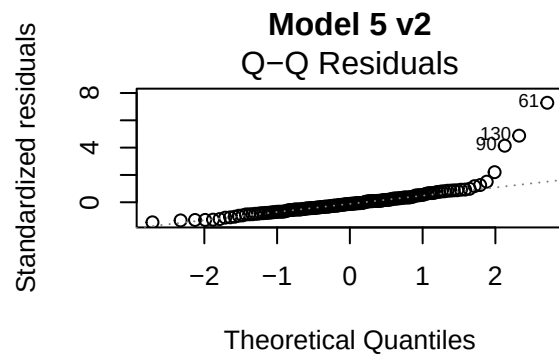
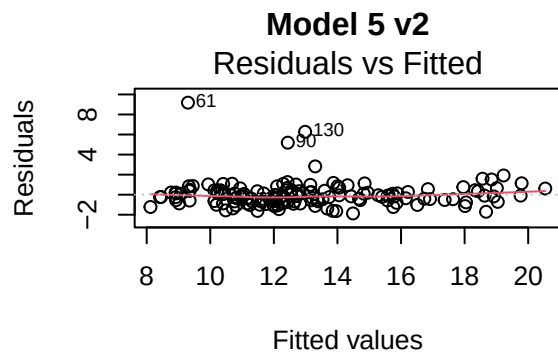
```
plot(model.3, main = "Model 3 v2")
```



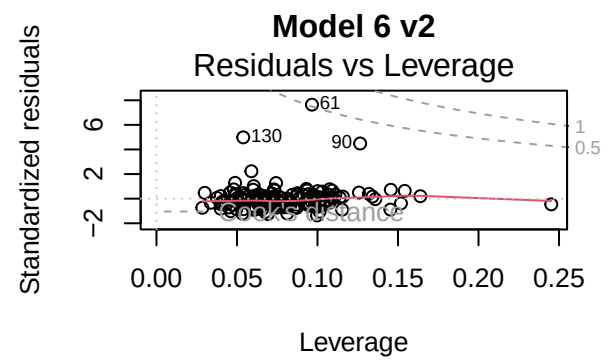
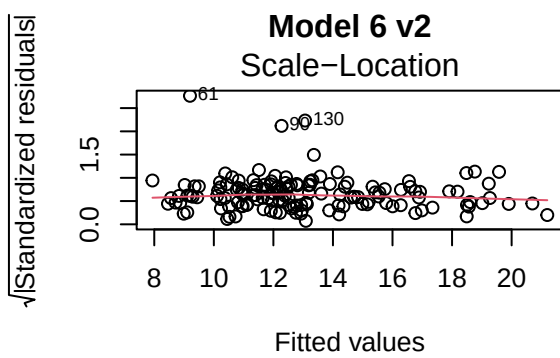
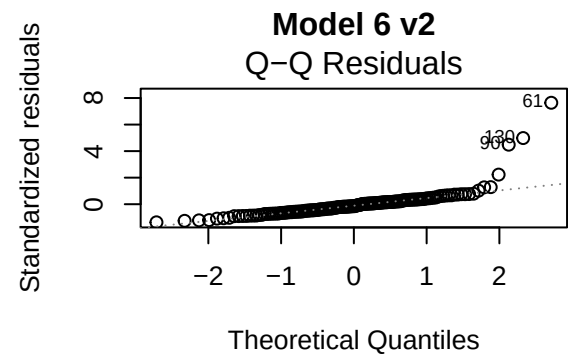
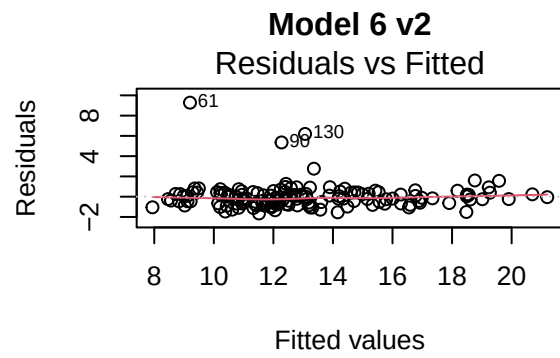
```
plot(model.4, main = "Model 4 v2")
```



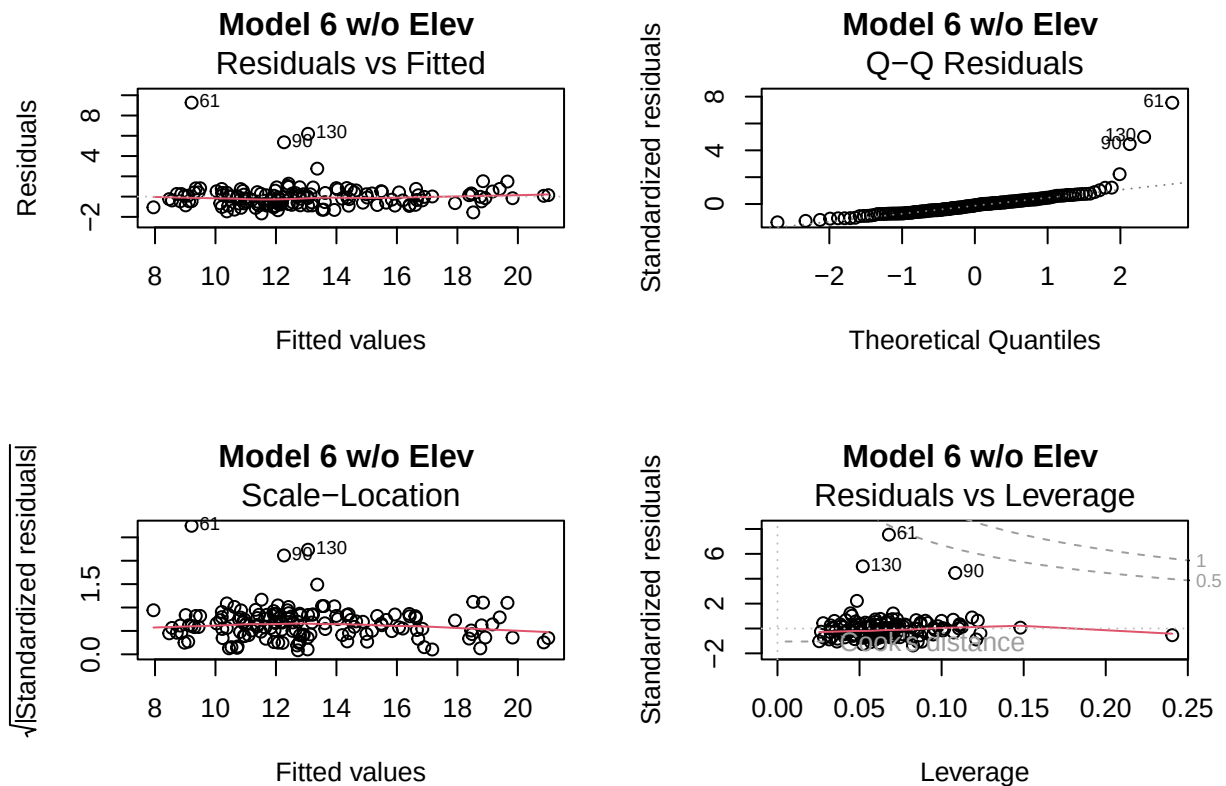
```
plot(model.5, main = "Model 5 v2")
```



```
plot(model.6, main = "Model 6 v2")
```



```
plot(model.6_noElev, main = "Model 6 w/o Elev ")
```



More or less, these models look the same. Still have irregularity in normality at upper thresholds.

```
#hist(full_ungulates$ndvi)
```

Finally noticed the normality issues EXCEPT, now that I'm actually looking at the normality more closely, we have some problems.

These are shapiro.test() 's on the models

```
shapiro.test(model.2$residuals)
```

```
##
## Shapiro-Wilk normality test
##
## data: model.2$residuals
## W = 0.84131, p-value = 1.917e-11
```

```
shapiro.test(model.3$residuals)
```

```
##
## Shapiro-Wilk normality test
##
## data: model.3$residuals
## W = 0.76523, p-value = 3.233e-14
```

```
shapiro.test(model.4$residuals)
```

```
##  
## Shapiro-Wilk normality test  
##  
## data:  model.4$residuals  
## W = 0.76205, p-value = 2.561e-14
```

```
shapiro.test(model.5$residuals)
```

```
##  
## Shapiro-Wilk normality test  
##  
## data:  model.5$residuals  
## W = 0.69883, p-value = 3.724e-16
```

```
shapiro.test(model.6$residuals)
```

```
##  
## Shapiro-Wilk normality test  
##  
## data:  model.6$residuals  
## W = 0.63593, p-value < 2.2e-16
```

Noticed that NDVI, dist_water, and human_footprint are not normally distributed. That's not actually what's important here, it's the model residuals that need to be normal. And those are a problem here.

So, what now? Well I've finally figured out how to interpret the models, and I've figured out how to correctly set them up.

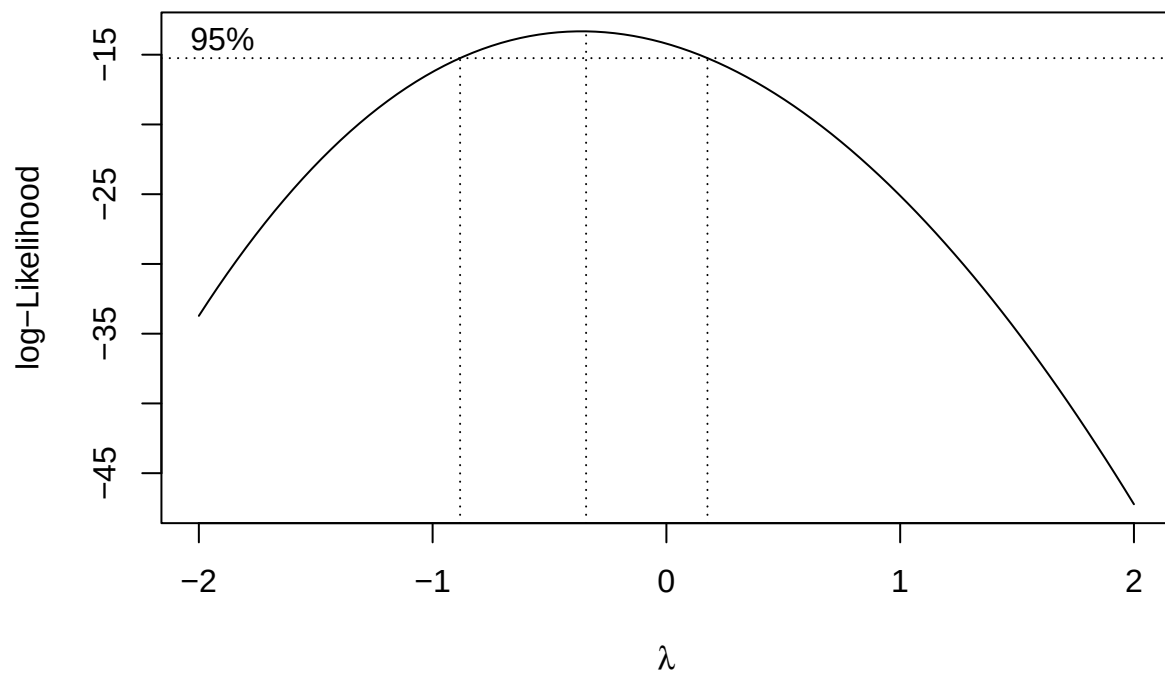
Given the normality issues (and no other violations by the look of the plots), I think I need to try some transformations.

Transformations

Start with a boxcox():

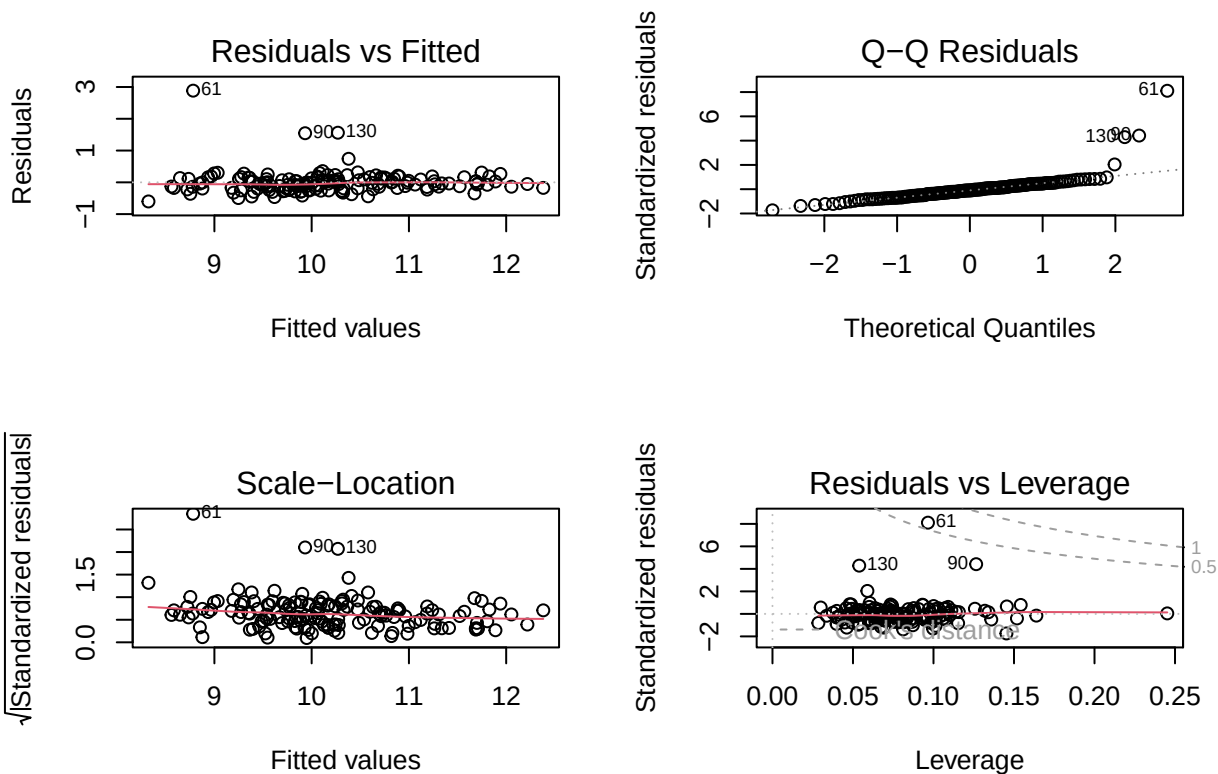
```
library(MASS)
```

```
boxcox(model.6)
```



Going to start with $\lambda = 0.25$

```
log.lm = lm(log(density, base = exp(0.25)) ~ season * (ndvi + dist_water + elevation + human_footprint + ...
par(mfrow = c(2,2))
plot(log.lm)
```



```
shapiro.test(log.lm$residuals)
```

```
##
##  Shapiro-Wilk normality test
##
## data:  log.lm$residuals
## W = 0.64063, p-value < 2.2e-16
```

Still a problem. Am noticing theres a few points that are outliers in all the diagnostics plots.

I'm going to run cooks distance and try to id them.

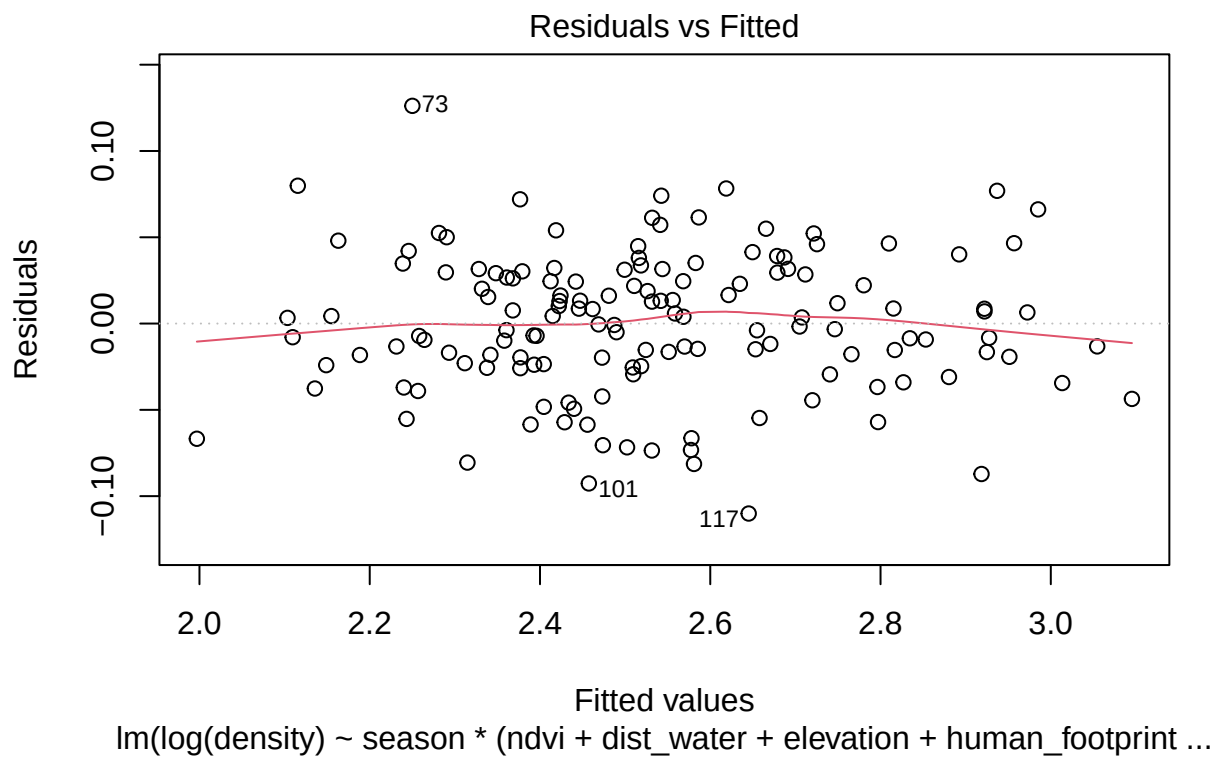
```
# running cooks distance on model.6
CD.model.6 <- cooks.distance(model.6)
CD.crit <- 3*mean(CD.model.6)
full_ungulates[which(CD.model.6 > CD.crit),]
```

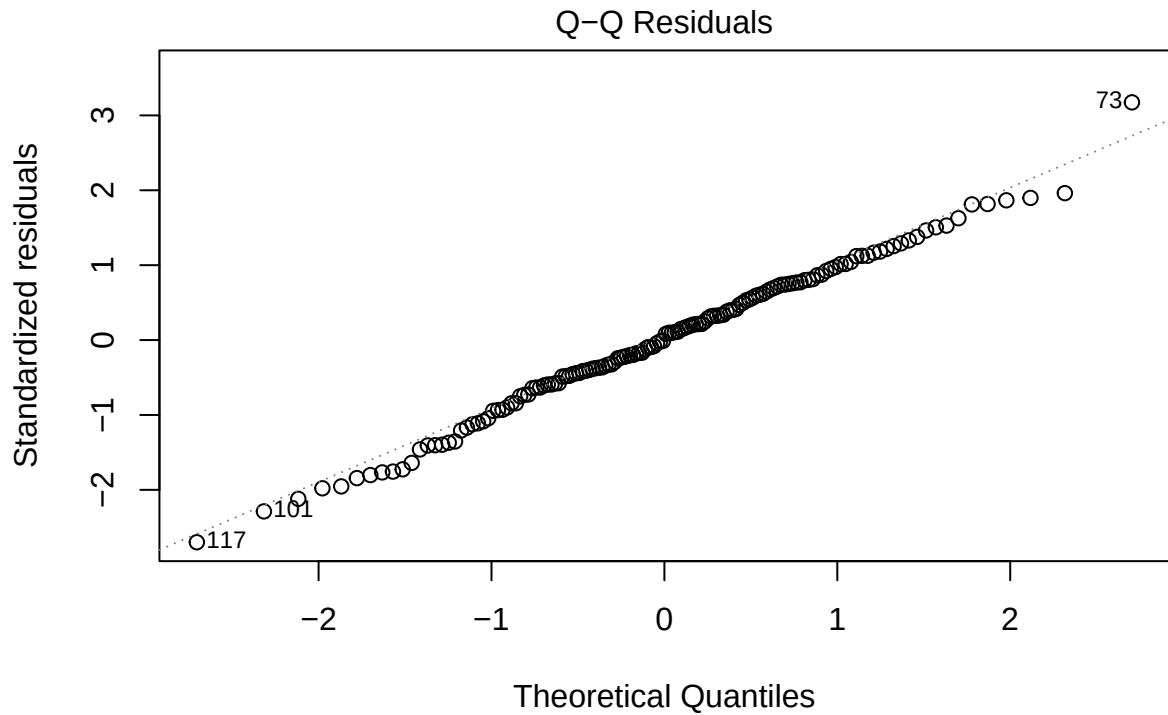
```
##          ndvi dist_water elevation human_footprint snow_depth density region
## 39  0.8254298  1.868626 1807.2166      50.39939    8.197178 16.12204  North
## 61  0.3733503  4.613136 2213.6940      89.73185   45.934787 18.47730  South
## 90  0.8097978  4.411319  383.4791      74.64486   34.598064 17.62848  Central
## 130 0.7840584  1.587397  988.2534      72.74659    9.559316 19.25493  North
##      season
## 39  Winter
## 61  Winter
```

```
## 90 Winter  
## 130 Winter
```

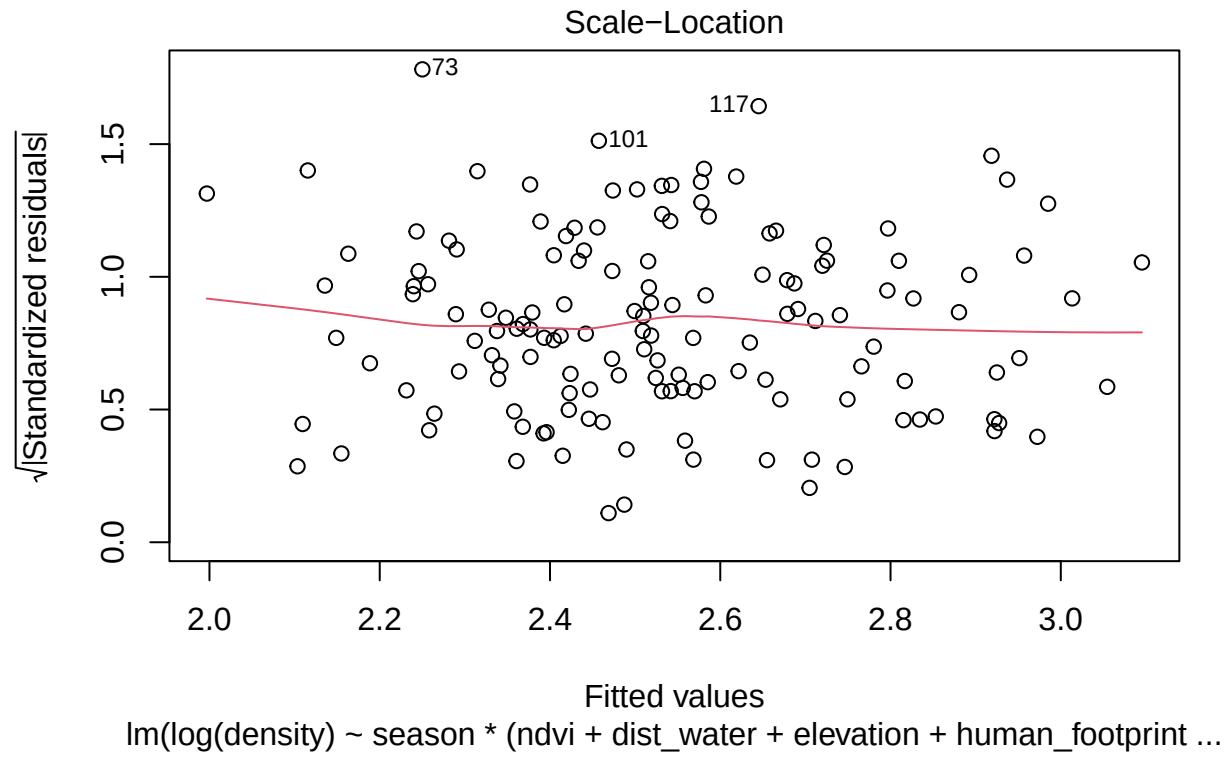
The points that show up here seem to be ones that show up in that diagnostics too

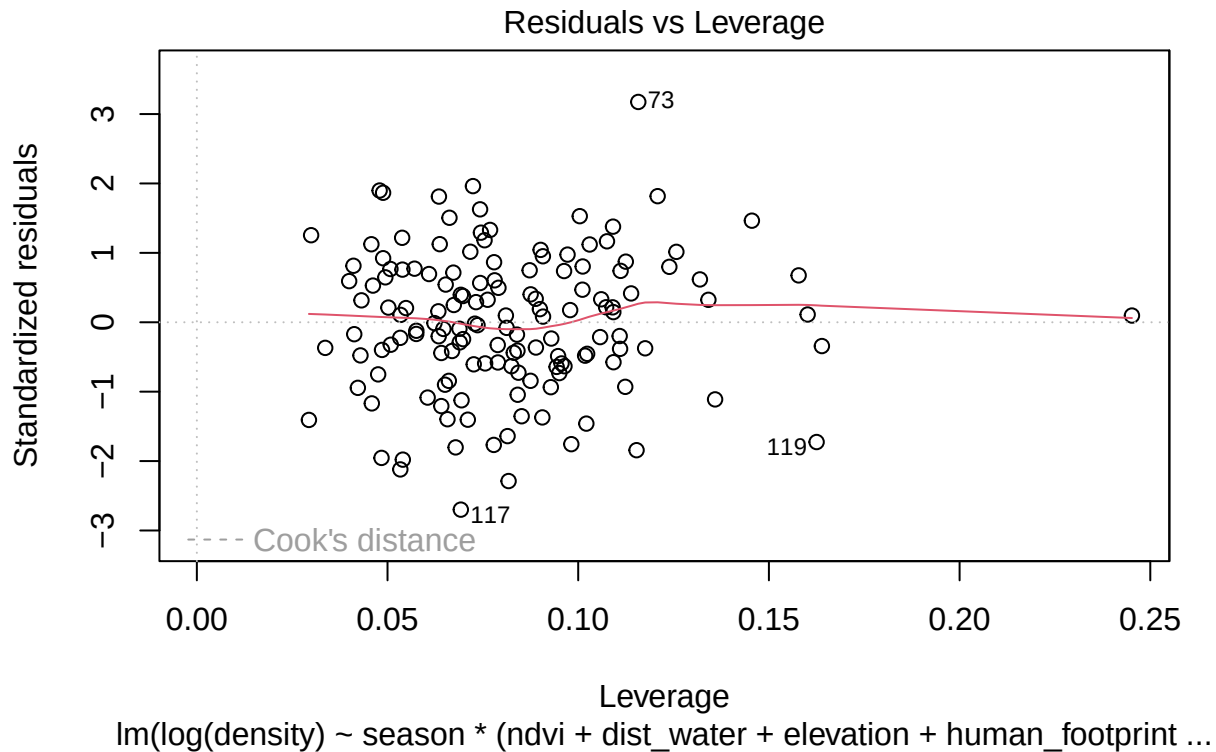
```
# If remove these points, what happens?  
model.6_woPoints <- lm(log(density) ~ season *  
  (ndvi + dist_water + elevation + human_footprint + snow_depth),  
  data = full_ungulates[-c(39, 61, 90, 130), ])  
  
plot(model.6_woPoints)
```





$\ln(\log(\text{density})) \sim \text{season} * (\text{ndvi} + \text{dist_water} + \text{elevation} + \text{human_footprint} \dots)$





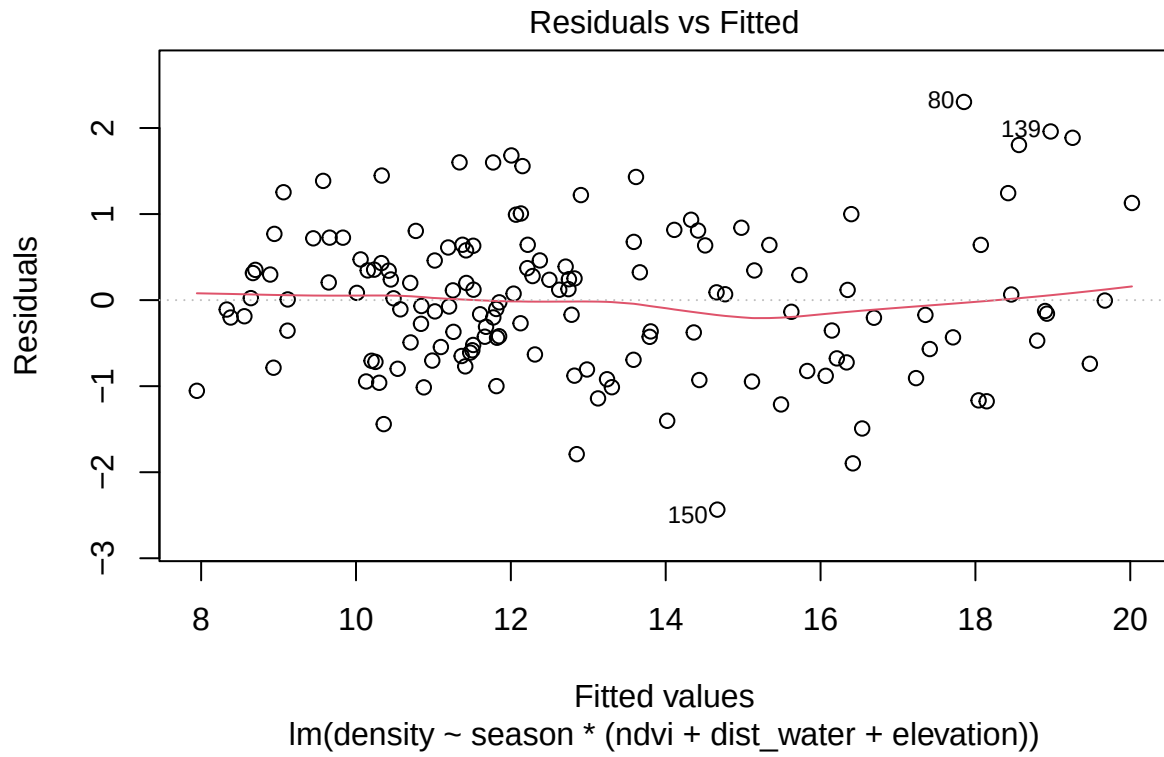
```
shapiro.test(model.6_woPoints$residuals)
```

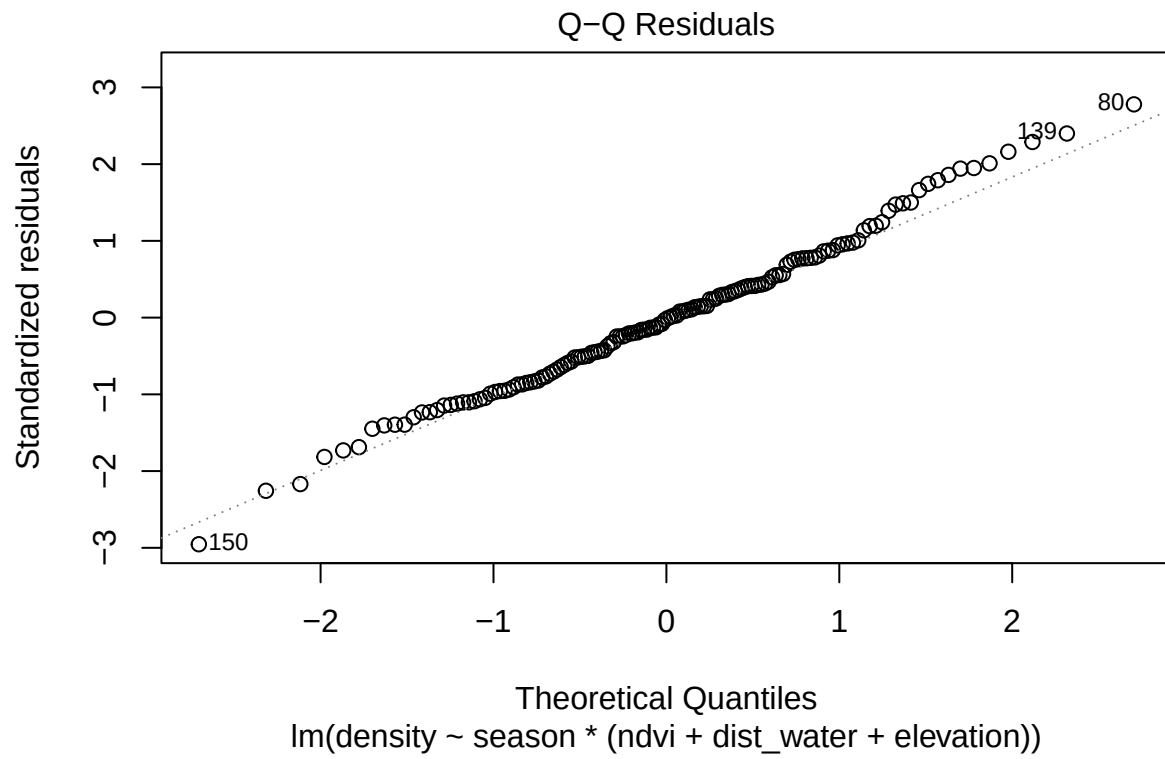
```
##
##  Shapiro-Wilk normality test
##
## data:  model.6_woPoints$residuals
## W = 0.99501, p-value = 0.9002
```

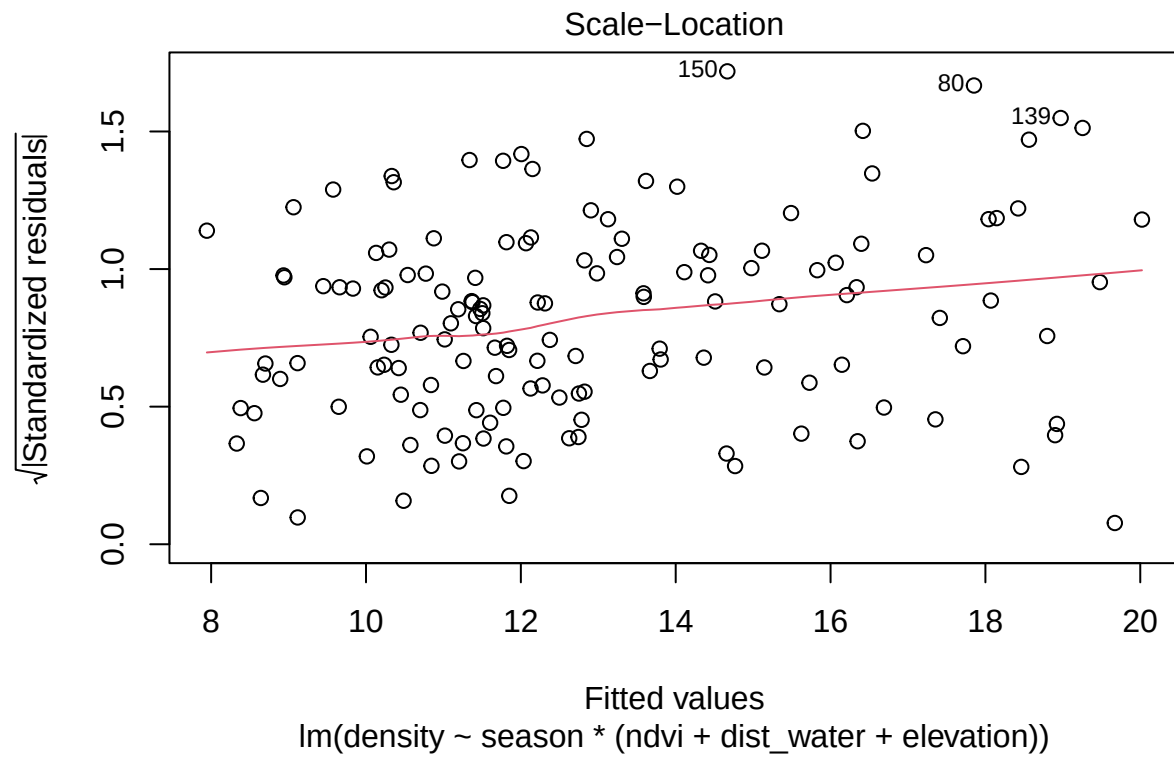
```
ncvTest(model.6_woPoints)
```

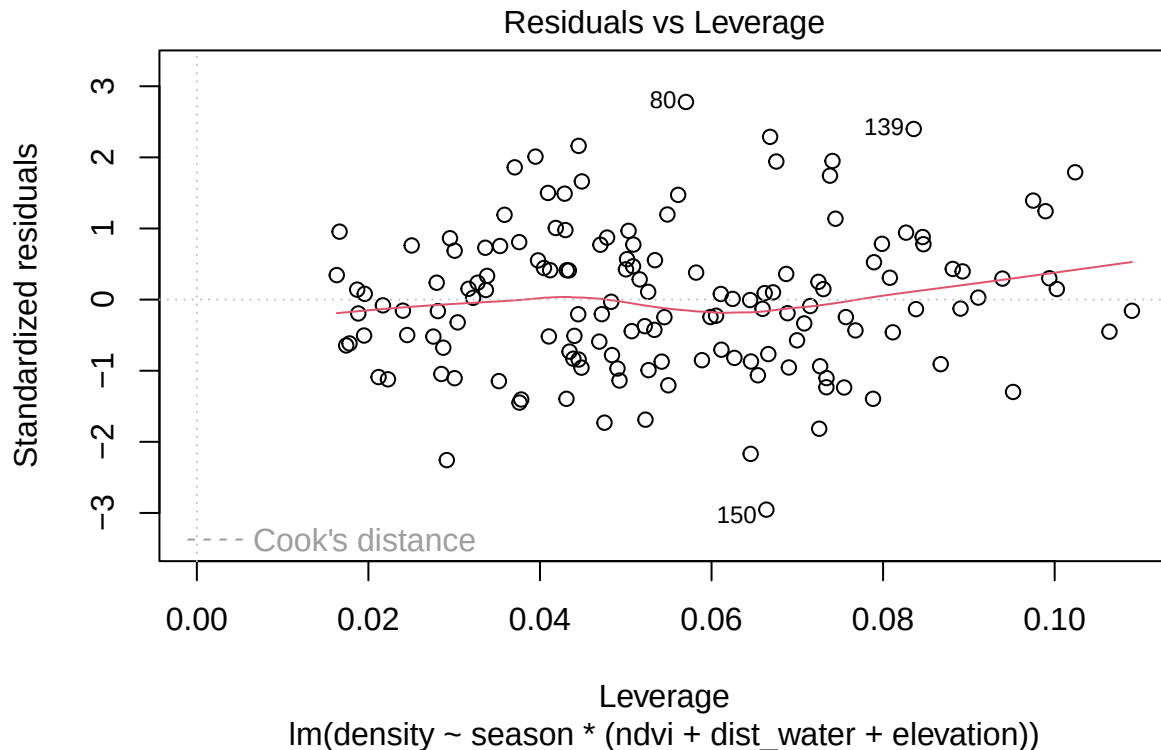
```
## Non-constant Variance Score Test
## Variance formula: ~ fitted.values
## Chisquare = 0.1825325, Df = 1, p = 0.66921
```

```
model.4_woPoints <- lm(density ~ season *
                        (ndvi + dist_water + elevation),
                        data = full_ungulates[-c(39, 61, 90, 130), ])
plot(model.4_woPoints)
```









```
shapiro.test(model.4_woPoints$residuals)
```

```
##
##  Shapiro-Wilk normality test
##
## data:  model.4_woPoints$residuals
## W = 0.9927, p-value = 0.663
```

```
ncvTest(model.4_woPoints)
```

```
## Non-constant Variance Score Test
## Variance formula: ~ fitted.values
## Chisquare = 11.40883, Df = 1, p = 0.00073096
```

So after removing those 4 points, ran tests with model.4 and model.6 to check the normality. In both cases, normality was fixed. Now that they gone, saw some heteroscedasticity on the residuals. Applied log on model.6, didn't on 4, did some trial and error. And yeah, looks like need a log(density) transformation, and I need those datapoints are removed to get normality and homoscedasticity.

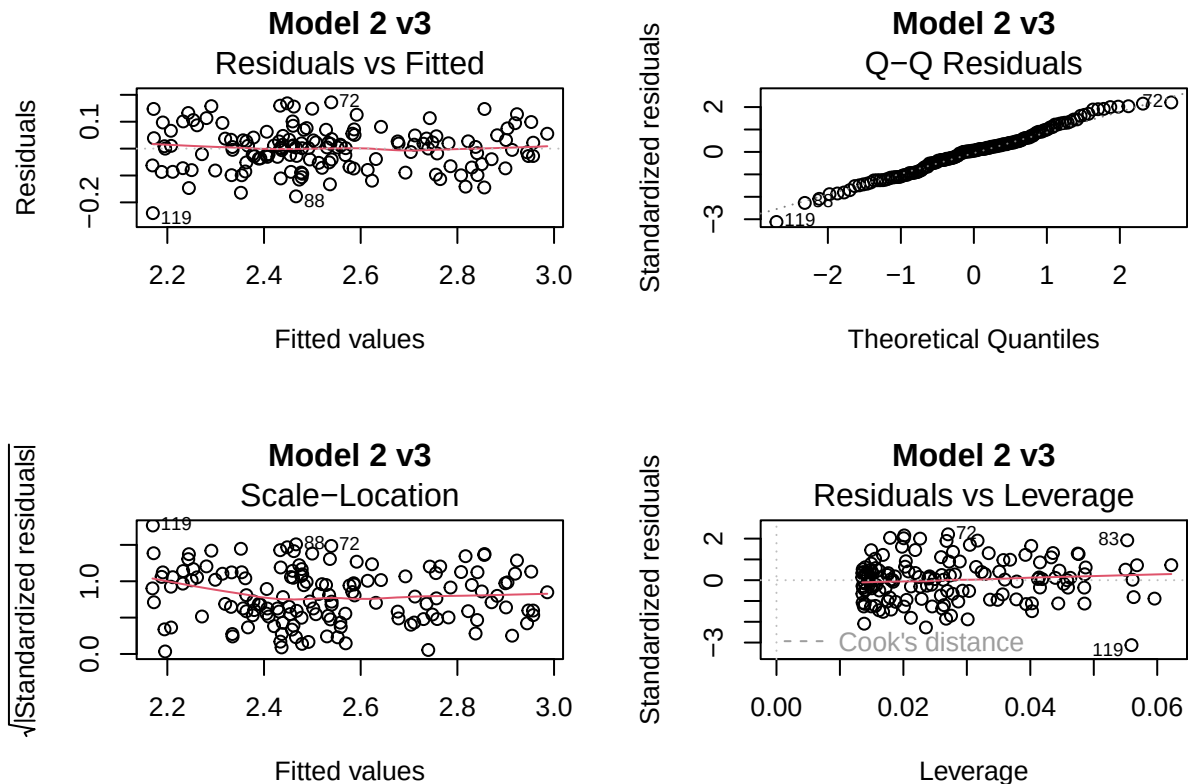
So now, will need to make the models again without the 4 points and with a log transform on density. Just going to use plain log(x) for simplicity, using log(x, base = -0.25) didn't change anything.

Just going to give them a different name from the originals for the Model Diagnostics assignment submission. But afterwards, I'm probably going to comment out the old models and reuse the 'model.#' format.

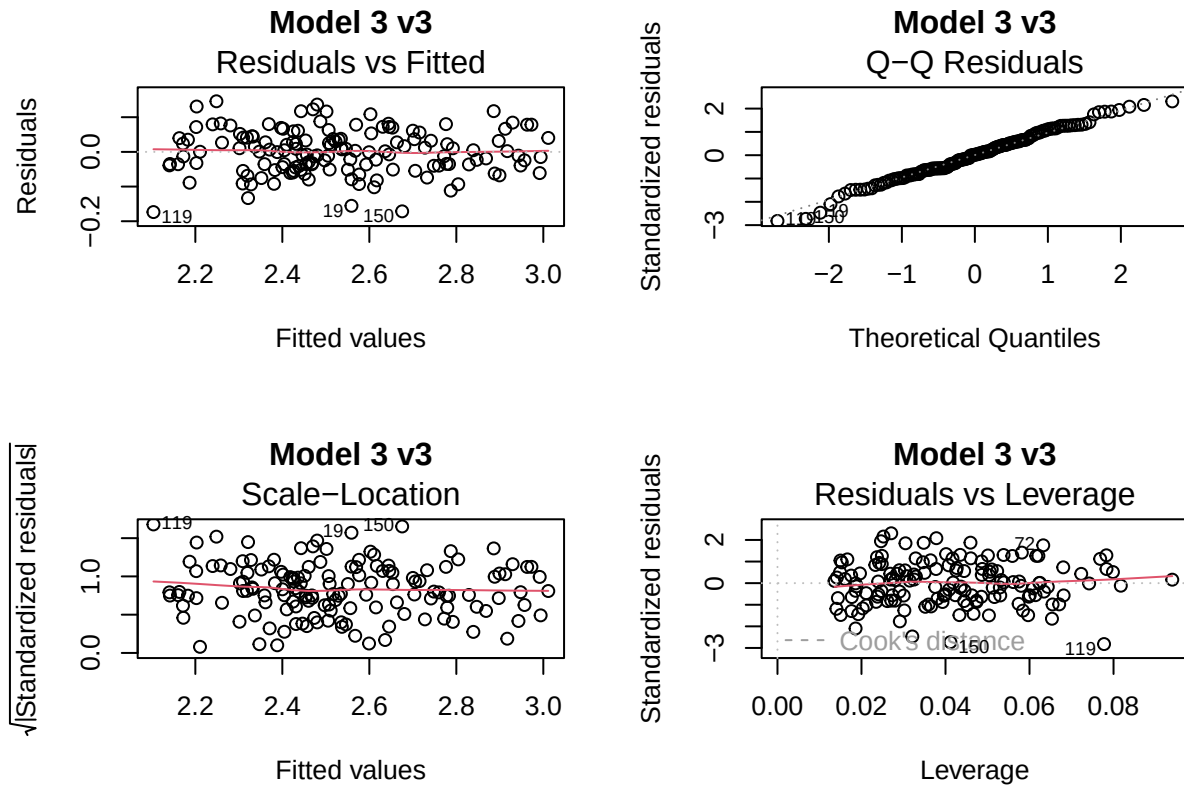
```
# Trial 3 on the models, without those 4 points and with log(density)

model.1.v3 = lm(log(density) ~ season, data = full_ungulates[-c(39, 61, 90, 130), ])
model.2.v3 = lm(log(density) ~ season * ndvi,
  data = full_ungulates[-c(39, 61, 90, 130), ])
model.3.v3 = lm(log(density) ~ season * (ndvi + dist_water),
  data = full_ungulates[-c(39, 61, 90, 130), ])
model.4.v3 = lm(log(density) ~ season * (ndvi + dist_water + elevation),
  data = full_ungulates[-c(39, 61, 90, 130), ])
model.5.v3 = lm(log(density) ~ season * (ndvi + dist_water + elevation
  + human_footprint),
  data = full_ungulates[-c(39, 61, 90, 130), ])
model.6.v3 = lm(log(density) ~ season *
  (ndvi + dist_water + elevation + human_footprint + snow_depth),
  data = full_ungulates[-c(39, 61, 90, 130), ])
```

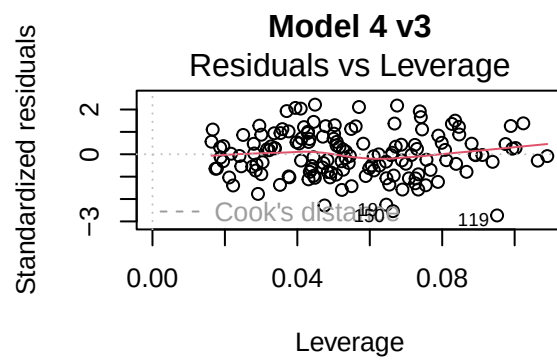
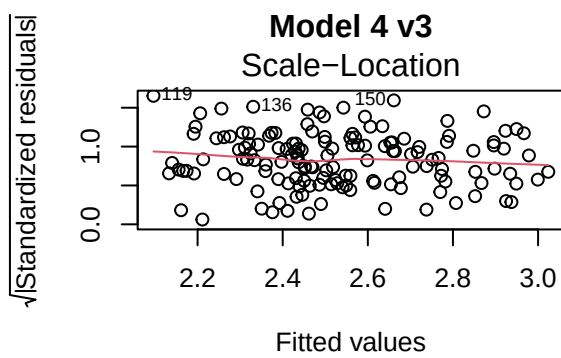
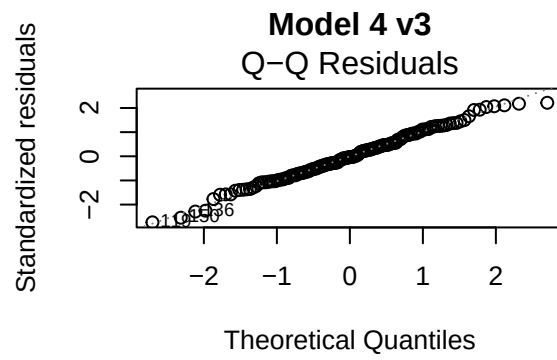
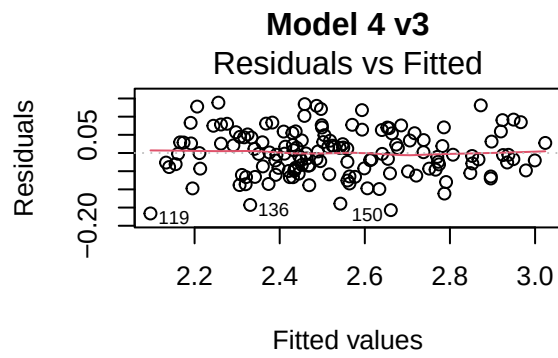
```
# Diagnostics on v3 models
par(mfrow = c(2,2))
plot(model.2.v3, main = "Model 2 v3")
```



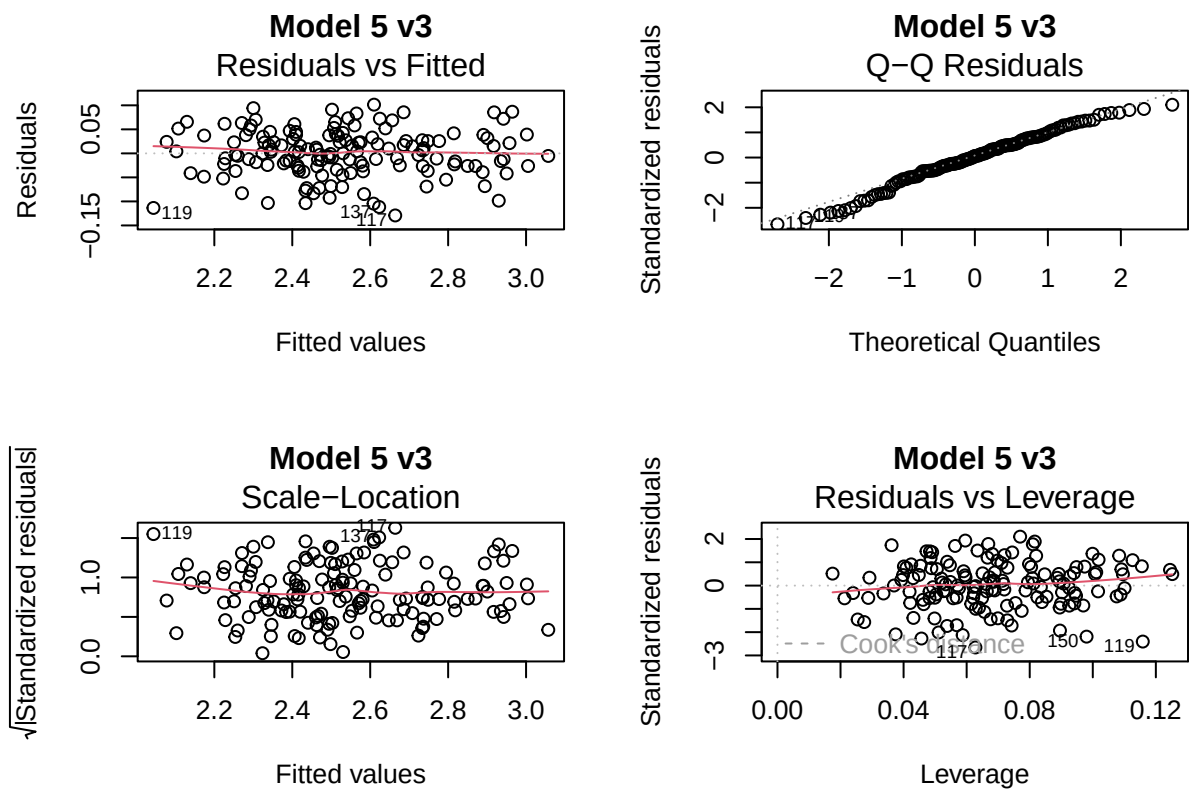
```
plot(model.3.v3, main = "Model 3 v3")
```



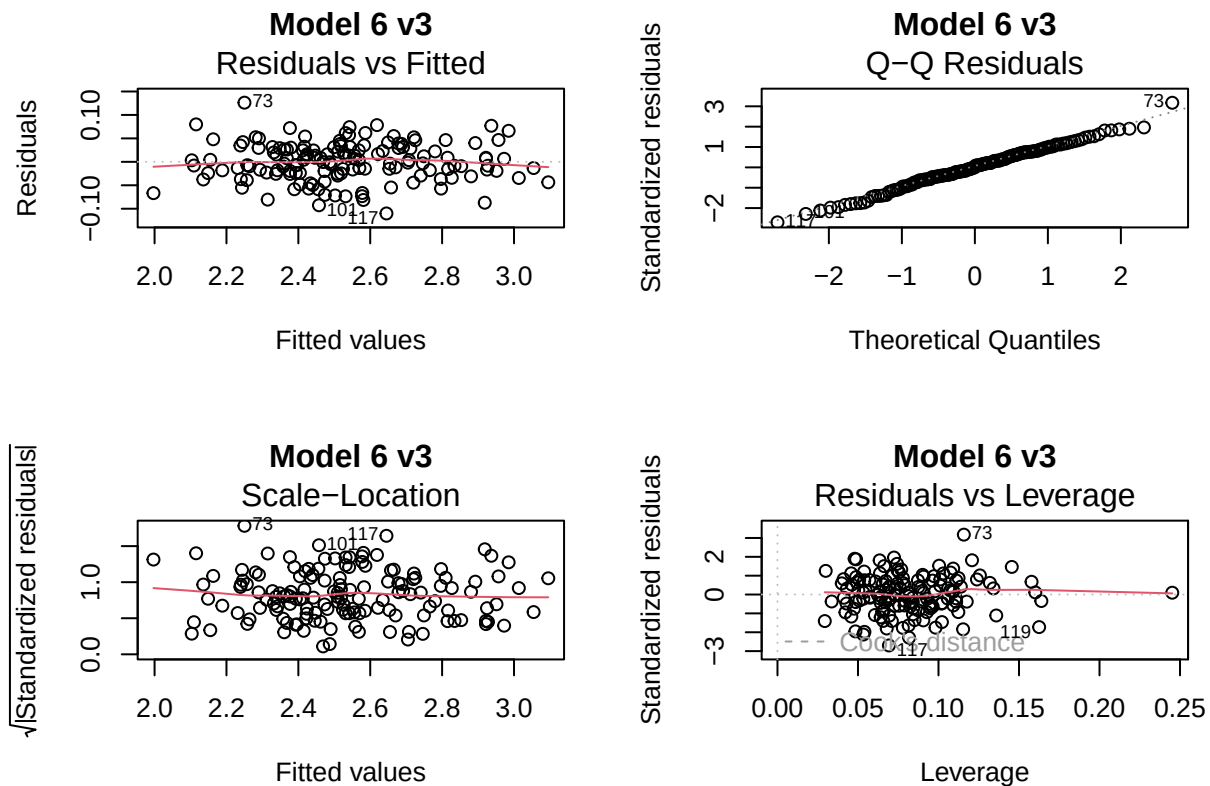
```
plot(model.4.v3, main = "Model 4 v3")
```



```
plot(model.5.v3, main = "Model 5 v3")
```



```
plot(model.6.v3, main = "Model 6 v3")
```



Overall diagnostics look better for these models now.

Model Selection, hopefully

Now that the models are in a better place, can start doing some comparison work. Run prediction code, decide which one(s) are best.

Probably should just make `full_ungulates = full_ungulates[-c(39, 61, 90, 130), ]`. Part of the data cleaning.

```
clean_ungulates = full_ungulates[-c(39, 61, 90, 130), ]
```

Last check of variance:

```
ncvTest(model.6.v3, ~ ndvi)
```

```
## Non-constant Variance Score Test
## Variance formula: ~ ndvi
## Chisquare = 0.0001388811, Df = 1, p = 0.9906
```

```
ncvTest(model.6.v3, ~ dist_water)
```

```
## Non-constant Variance Score Test
## Variance formula: ~ dist_water
## Chisquare = 0.4830033, Df = 1, p = 0.48707
```

```
ncvTest(model.6.v3, ~ elevation)
```

```
## Non-constant Variance Score Test
## Variance formula: ~ elevation
## Chisquare = 1.102732, Df = 1, p = 0.29367
```

```
ncvTest(model.6.v3, ~ human_footprint)
```

```
## Non-constant Variance Score Test
## Variance formula: ~ human_footprint
## Chisquare = 0.07478847, Df = 1, p = 0.78449
```

```
ncvTest(model.6.v3, ~ snow_depth)
```

```
## Non-constant Variance Score Test
## Variance formula: ~ snow_depth
## Chisquare = 5.256818, Df = 1, p = 0.021861
```

All the variables look good for homoscedasticity, except for snow_depth. Will have to see what I do about that.

```
summary(model.1.v3)
```

```
##
## Call:
## lm(formula = log(density) ~ season, data = full_ungulates[-c(39,
##    61, 90, 130), ])
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.43914 -0.12811  0.00903  0.09788  0.35331
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   2.69834    0.01902   141.86 <2e-16 ***
## seasonWinter -0.32902    0.02672   -12.31 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.1614 on 144 degrees of freedom
## Multiple R-squared:  0.5129, Adjusted R-squared:  0.5096
## F-statistic: 151.7 on 1 and 144 DF, p-value: < 2.2e-16
```

```
summary(model.2.v3)
```

```
##
## Call:
## lm(formula = log(density) ~ season * ndvi, data = full_ungulates[-c(39,
##    61, 90, 130), ])
##
##
```

```
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.239875 -0.052504  0.004327  0.044685  0.172095
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      2.27265     0.02598  87.492 < 2e-16 ***
## seasonWinter     -0.23041     0.03839  -6.001 1.55e-08 ***
## ndvi              0.80920     0.04609  17.557 < 2e-16 ***
## seasonWinter:ndvi -0.20495     0.06756  -3.034 0.00287 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.07908 on 142 degrees of freedom
## Multiple R-squared:  0.8847, Adjusted R-squared:  0.8823
## F-statistic: 363.2 on 3 and 142 DF,  p-value: < 2.2e-16
```

```
summary(model.3.v3)
```

```
##
## Call:
## lm(formula = log(density) ~ season * (ndvi + dist_water), data = full_ungulates[-c(39,
##      61, 90, 130), ])
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.173549 -0.039985  0.001301  0.040924  0.145895
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      2.351456     0.025928  90.691 < 2e-16 ***
## seasonWinter     -0.201982     0.037830  -5.339 3.68e-07 ***
## ndvi              0.794048     0.037463  21.196 < 2e-16 ***
## dist_water       -0.027509     0.005285  -5.205 6.76e-07 ***
## seasonWinter:ndvi -0.215479     0.054945  -3.922 0.000137 ***
## seasonWinter:dist_water -0.007419     0.007263  -1.021 0.308816
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.06408 on 140 degrees of freedom
## Multiple R-squared:  0.9254, Adjusted R-squared:  0.9227
## F-statistic: 347.1 on 5 and 140 DF,  p-value: < 2.2e-16
```

```
summary(model.4.v3)
```

```
##
## Call:
## lm(formula = log(density) ~ season * (ndvi + dist_water + elevation),
##      data = full_ungulates[-c(39, 61, 90, 130), ])
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.166168 -0.042954 -0.002207  0.042997  0.138431
```

```
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      2.377e+00  3.138e-02  75.744 < 2e-16 ***
## seasonWinter     -2.138e-01  4.444e-02  -4.811 3.89e-06 ***
## ndvi              7.891e-01  3.752e-02  21.030 < 2e-16 ***
## dist_water       -2.692e-02  5.286e-03  -5.094 1.13e-06 ***
## elevation        -1.688e-05  1.188e-05  -1.421 0.157703
## seasonWinter:ndvi -2.121e-01  5.493e-02  -3.861 0.000173 ***
## seasonWinter:dist_water -7.596e-03  7.269e-03  -1.045 0.297880
## seasonWinter:elevation  6.315e-06  1.693e-05   0.373 0.709749
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.0639 on 138 degrees of freedom
## Multiple R-squared:  0.9268, Adjusted R-squared:  0.9231
## F-statistic: 249.7 on 7 and 138 DF,  p-value: < 2.2e-16
```

```
summary(model.5.v3)
```

```
##
## Call:
## lm(formula = log(density) ~ season * (ndvi + dist_water + elevation +
##   human_footprint), data = full_ungulates[-c(39, 61, 90, 130),
##   ])
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.129219 -0.026279  0.002869  0.034297  0.101393
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      2.442e+00  2.710e-02  90.120 < 2e-16 ***
## seasonWinter     -1.884e-01  3.878e-02  -4.857 3.23e-06 ***
## ndvi              7.815e-01  2.953e-02  26.464 < 2e-16 ***
## dist_water       -2.753e-02  4.157e-03  -6.622 7.56e-10 ***
## elevation        -1.814e-05  9.343e-06  -1.941  0.0543 .
## human_footprint   -1.126e-03  1.921e-04  -5.862 3.30e-08 ***
## seasonWinter:ndvi -2.279e-01  4.333e-02  -5.260 5.46e-07 ***
## seasonWinter:dist_water -8.226e-03  5.719e-03  -1.438  0.1526
## seasonWinter:elevation  2.437e-06  1.333e-05   0.183  0.8552
## seasonWinter:human_footprint -2.386e-04  2.685e-04  -0.889  0.3758
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.05024 on 136 degrees of freedom
## Multiple R-squared:  0.9554, Adjusted R-squared:  0.9525
## F-statistic: 323.9 on 9 and 136 DF,  p-value: < 2.2e-16
```

```
summary(model.6.v3)
```

```
##
## Call:
```

```
## lm(formula = log(density) ~ season * (ndvi + dist_water + elevation +
##      human_footprint + snow_depth), data = full_ungulates[-c(39,
##      61, 90, 130), ])
##
## Residuals:
##      Min        1Q      Median        3Q        Max
## -0.110075 -0.024035  0.001402  0.029434  0.126154
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      2.615e+00  3.820e-02  68.466 < 2e-16 ***
## seasonWinter     -2.122e-01  5.338e-02  -3.975 0.000114 ***
## ndvi              6.403e-01  3.525e-02  18.166 < 2e-16 ***
## dist_water       -2.864e-02  3.503e-03  -8.176 1.97e-13 ***
## elevation        -2.168e-05  7.885e-06  -2.750 0.006779 **
## human_footprint  -1.173e-03  1.618e-04  -7.247 3.04e-11 ***
## snow_depth       -3.017e-03  5.345e-04  -5.644 9.50e-08 ***
## seasonWinter:ndvi -2.155e-01  5.083e-02  -4.240 4.13e-05 ***
## seasonWinter:dist_water -5.090e-03  4.831e-03  -1.054 0.293926
## seasonWinter:elevation  1.319e-06  1.127e-05   0.117 0.906998
## seasonWinter:human_footprint -2.063e-04  2.261e-04  -0.913 0.363049
## seasonWinter:snow_depth  2.830e-04  7.547e-04   0.375 0.708243
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.04226 on 134 degrees of freedom
## Multiple R-squared:  0.9689, Adjusted R-squared:  0.9664
## F-statistic: 379.8 on 11 and 134 DF,  p-value: < 2.2e-16
```

Looking at all summary() outputs, can see that the estimates look very different after applying the log transformation. More importantly, the significance of the estimates appears to be more confident. Now, even elevation in the summer is considered significant in model 6.3.v3, thought only after snow_depth is included. The significance level of ndvi in the winter has increased, thought it is the only predictor to significantly change from summer to winter.

Unfortunately as mentioned, snow_depth appears to have a homogeneity problem. So need to decide what to do with it, if can't fix it (have already transformed the response).

Note that even with just two to three predictors, R^2 has improved quite dramatically.

```
#elg = lm(log(density) ~ season *
#      (ndvi + dist_water + elevation + human_footprint + snow_depth + #I(snow_depth^2)
#      data = full_ungulates[-c(39, 61, 90, 130), ])
#summary(elg)
```

```
#ncvTest(elg, ~ ndvi)
#ncvTest(elg, ~ dist_water)
#ncvTest(elg, ~ elevation)
#ncvTest(elg, ~ human_footprint)
#ncvTest(elg, ~ snow_depth)
#ncvTest(elg, ~ I(snow_depth^2))
```

The above two chunks were attempts to fix the heterogeneity of snow_depth by using $I(\text{snow_depth}^2)$. Doing so worked, although only with $I(\text{snow_depth}^2)$. Adding in snow_depth (so running x and x^2 together) or using poly() didn't work.

Comparing the models

```
library(lmtest)
```

```
## Loading required package: zoo
```

```
##
```

```
## Attaching package: 'zoo'
```

```
## The following objects are masked from 'package:base':
```

```
##
```

```
##      as.Date, as.Date.numeric
```

Ok enough of that. Run some AICc and `lrtest()` stuff and lets see how the models compare

```
# lrtest model 1 v 2
```

```
lrtest(model.1.v3, model.2.v3)
```

```
## Likelihood ratio test
```

```
##
```

```
## Model 1: log(density) ~ season
```

```
## Model 2: log(density) ~ season * ndvi
```

```
##   #Df LogLik Df  Chisq Pr(>Chisq)
```

```
## 1    3  60.125
```

```
## 2    5 165.314  2 210.38  < 2.2e-16 ***
```

```
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

`lrtest()`, getting a significant p-value means reject null hypothesis that the simpler model is as good or better than the complex model. So here, model.2.v3 has an improved likelihood (as expected).

```
# lrtest model 2 v 3
```

```
lrtest(model.2.v3, model.3.v3)
```

```
## Likelihood ratio test
```

```
##
```

```
## Model 1: log(density) ~ season * ndvi
```

```
## Model 2: log(density) ~ season * (ndvi + dist_water)
```

```
##   #Df LogLik Df  Chisq Pr(>Chisq)
```

```
## 1    5 165.31
```

```
## 2    7 197.05  2  63.473  1.648e-14 ***
```

```
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

So, model.3.v3 has an improved likelihood over model.2.v3

```
# lrtest model 3 v 4
```

```
lrtest(model.3.v3, model.4.v3)
```

```
## Likelihood ratio test
##
## Model 1: log(density) ~ season * (ndvi + dist_water)
## Model 2: log(density) ~ season * (ndvi + dist_water + elevation)
##   #Df LogLik Df  Chisq Pr(>Chisq)
## 1    7 197.05
## 2    9 198.51  2 2.9164    0.2326
```

Elevation was not significant until model6 usually. That's confirmed here, don't reject null hypo that model.3.v3 is as good as model.4.v3

```
# lrtest model 3 v 5
lrtest(model.3.v3, model.5.v3)
```

```
## Likelihood ratio test
##
## Model 1: log(density) ~ season * (ndvi + dist_water)
## Model 2: log(density) ~ season * (ndvi + dist_water + elevation + human_footprint)
##   #Df LogLik Df  Chisq Pr(>Chisq)
## 1    7 197.05
## 2   11 234.69  4 75.277 1.741e-15 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Significant p-value, so conclude that more complex model.5.v3 has higher likelihood than model.3.v3

```
# lrtest model 5 v 6
lrtest(model.5.v3, model.6.v3)
```

```
## Likelihood ratio test
##
## Model 1: log(density) ~ season * (ndvi + dist_water + elevation + human_footprint)
## Model 2: log(density) ~ season * (ndvi + dist_water + elevation + human_footprint +
##   snow_depth)
##   #Df LogLik Df  Chisq Pr(>Chisq)
## 1   11 234.69
## 2   13 261.01  2 52.647 3.698e-12 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Again get a significant p-value, so model.6.v3 was has the highest likelihood of the models.

```
# running dredge now

full.model = lm(log(density) ~ season *
                  (ndvi + dist_water + elevation + human_footprint + snow_depth),
                  data = clean_ungulates, na.action = "na.fail")
models = dredge(full.model)
```

```
## Fixed term is "(Intercept)"
```

```
head(models)
```

```
## Global model call: lm(formula = log(density) ~ season * (ndvi + dist_water + elevation +
##   human_footprint + snow_depth), data = clean_ungulates, na.action = "na.fail")
## ---
## Model selection table
##      (Int)  dst_wtr      elv   hmn_ftp   ndv  ssn   snw_dpt  dst_wtr:ssn
## 576  2.622 -0.03122 -2.102e-05 -0.001277 0.6432   + -0.002914
## 640  2.613 -0.02867 -2.095e-05 -0.001278 0.6461   + -0.002882      +
## 832  2.616 -0.03124 -2.114e-05 -0.001174 0.6440   + -0.002909
## 1600 2.630 -0.03129 -2.099e-05 -0.001278 0.6367   + -0.003054
## 704  2.622 -0.03122 -2.123e-05 -0.001277 0.6432   + -0.002914
## 896  2.607 -0.02860 -2.107e-05 -0.001170 0.6471   + -0.002875      +
##      elv:ssn hmn_ftp:ssn ndv:ssn ssn:snw_dpt df  logLik   AICc delta weight
## 576                                     +          9 259.919 -500.5  0.00  0.339
## 640                                     +          10 260.462 -499.3  1.22  0.184
## 832                        +          +          10 260.346 -499.1  1.45  0.164
## 1600                        +          +          + 10 259.993 -498.4  2.16  0.115
## 704              +          +          10 259.920 -498.2  2.30  0.107
## 896                        +          +          11 260.933 -497.9  2.62  0.091
## Models ranked by AICc(x)
```

So if a model has , that means the predictor isn't used. If it has '+' for a categorical var or interaction, means the predictor is included.

```
summary(model.6.v3)
```

```
##
## Call:
## lm(formula = log(density) ~ season * (ndvi + dist_water + elevation +
##   human_footprint + snow_depth), data = full_ungulates[-c(39,
##   61, 90, 130), ])
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.110075 -0.024035  0.001402  0.029434  0.126154
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    2.615e+00  3.820e-02  68.466 < 2e-16 ***
## seasonWinter   -2.122e-01  5.338e-02  -3.975 0.000114 ***
## ndvi           6.403e-01  3.525e-02  18.166 < 2e-16 ***
## dist_water     -2.864e-02  3.503e-03  -8.176 1.97e-13 ***
## elevation      -2.168e-05  7.885e-06  -2.750 0.006779 **
## human_footprint -1.173e-03  1.618e-04  -7.247 3.04e-11 ***
## snow_depth     -3.017e-03  5.345e-04  -5.644 9.50e-08 ***
## seasonWinter:ndvi -2.155e-01  5.083e-02  -4.240 4.13e-05 ***
## seasonWinter:dist_water -5.090e-03  4.831e-03  -1.054 0.293926
## seasonWinter:elevation  1.319e-06  1.127e-05   0.117 0.906998
## seasonWinter:human_footprint -2.063e-04  2.261e-04  -0.913 0.363049
## seasonWinter:snow_depth  2.830e-04  7.547e-04   0.375 0.708243
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.04226 on 134 degrees of freedom
## Multiple R-squared:  0.9689, Adjusted R-squared:  0.9664
## F-statistic: 379.8 on 11 and 134 DF,  p-value: < 2.2e-16
```

So according to the dredge function, the best model was pretty similar to model.6.v3 which I think is good.

```
AICc(model.6.v3) # same as full.model
```

```
## [1] -493.2681
```

```
best_aicc = models[1, "AICc"]
my_aicc = AICc(model.6.v3)
ER = exp((my_aicc - best_aicc) /2)
ER
```

```
## [1] 37.4551
```

So the best model from dredge() included the same predictors, but different estimates. And it did drastically better than I did. The best model from dredge() is 37.5x more likely to than my model.6.v3

My model was 16th on the list of top models.

```
my_model <- get.models(models, subset = AICc < -493)
my_model$`2048`
```

```
##
## Call:
## lm(formula = log(density) ~ dist_water + elevation + human_footprint +
##      ndvi + season + snow_depth + dist_water:season + elevation:season +
##      human_footprint:season + ndvi:season + season:snow_depth +
##      1, data = clean_ungulates, na.action = "na.fail")
##
## Coefficients:
##              (Intercept)              dist_water
##              2.615e+00              -2.864e-02
##              elevation              human_footprint
##              -2.168e-05              -1.173e-03
##              ndvi              seasonWinter
##              6.403e-01              -2.122e-01
##              snow_depth      dist_water:seasonWinter
##              -3.017e-03              -5.090e-03
##              elevation:seasonWinter human_footprint:seasonWinter
##              1.319e-06              -2.063e-04
##              ndvi:seasonWinter      seasonWinter:snow_depth
##              -2.155e-01              2.830e-04
```

Can get the estimates of the best model from dredge() in same way.

```
best_model = get.models(models, subset = AICc < -499)
#best_model$`576`
summary(best_model$`576`)
```

```
##
## Call:
## lm(formula = log(density) ~ dist_water + elevation + human_footprint +
##     ndvi + season + snow_depth + ndvi:season + 1, data = clean_ungulates,
##     na.action = "na.fail")
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.113161 -0.025217 -0.000457  0.029918  0.122836
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    2.622e+00  2.802e-02  93.562 < 2e-16 ***
## dist_water     -3.122e-02  2.385e-03 -13.087 < 2e-16 ***
## elevation      -2.102e-05  5.591e-06  -3.760  0.00025 ***
## human_footprint -1.277e-03  1.121e-04 -11.389 < 2e-16 ***
## ndvi           6.432e-01  3.007e-02  21.388 < 2e-16 ***
## seasonWinter   -2.218e-01  2.042e-02 -10.864 < 2e-16 ***
## snow_depth     -2.914e-03  3.732e-04  -7.808 1.31e-12 ***
## ndvi:seasonWinter -2.236e-01  3.588e-02  -6.232 5.23e-09 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.04196 on 138 degrees of freedom
## Multiple R-squared:  0.9685, Adjusted R-squared:  0.9669
## F-statistic: 605.2 on 7 and 138 DF, p-value: < 2.2e-16
```

BTW keep in mind I need to backtransform to get a model.

So now, consider all the predictors.

- dist_water makes ecological sense, animals need to have access to water and would plausibly remain in close proximity to water
- elevation could be plausible for ungulates. Depending on how drastic the change is, plant variety will vary at lower versus higher altitudes which could determine how stable the feeding grounds are, especially for a herd
- human_footprint makes sense, less human disturbance and interference is likely to have a positive correlation with density and vice versa
- ndvi, if theres greenspace and foraging food then theres a basis for life
- seasonWinter, in winter theres a default decrease in density. That makes sense
- snow_depth, if theres more snow build up then animals in general will use more energy for moving about. no different for ungulates, and food becomes scarcer. makes sense
- ndvi:seasonwinter, in winter theres less greenspace than there is during summer, so lower populations hanging around and less density. makes sense

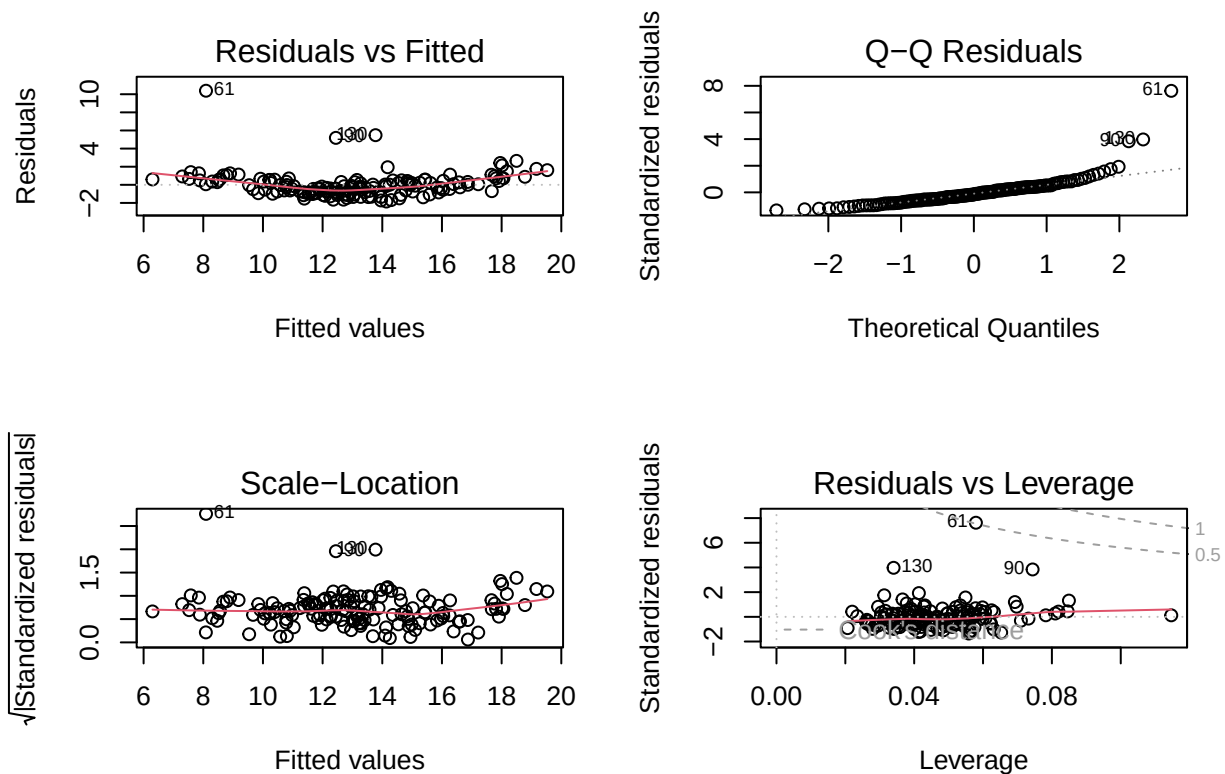
So all the predictors in this model seem to make sense.

Comparing Model Types

Earlier in this assignment I created ‘model_6’, which was a purely additive model. It was essentially a “Different Intercepts, Same Slopes” model. Having gone through all this work for a “Different Intercepts, Different Slopes” model, the best model from dredge() and my own build-up only has NDVI changing based on season. So which model is better?

I’ll start with an lrttest on that ‘model_6’ versus my newest ‘model.6.v3’. But first, check the diagnostics.

```
par(mfrow = c(2,2))
plot(model_6)
```



```
shapiro.test(model_2$residuals)
```

```
##
##  Shapiro-Wilk normality test
##
## data:  model_2$residuals
## W = 0.87148, p-value = 4.329e-10
```

```
shapiro.test(model_3$residuals)
```

```
##
##  Shapiro-Wilk normality test
```

```
##  
## data: model_3$residuals  
## W = 0.81706, p-value = 2.063e-12
```

```
shapiro.test(model_4$residuals)
```

```
##  
## Shapiro-Wilk normality test  
##  
## data: model_4$residuals  
## W = 0.81406, p-value = 1.589e-12
```

```
shapiro.test(model_5$residuals)
```

```
##  
## Shapiro-Wilk normality test  
##  
## data: model_5$residuals  
## W = 0.76103, p-value = 2.377e-14
```

```
shapiro.test(model_6$residuals)
```

```
##  
## Shapiro-Wilk normality test  
##  
## data: model_6$residuals  
## W = 0.72108, p-value = 1.524e-15
```

```
ncvTest(model_2)
```

```
## Non-constant Variance Score Test  
## Variance formula: ~ fitted.values  
## Chisquare = 0.6193177, Df = 1, p = 0.4313
```

```
cat("\n")
```

```
ncvTest(model_3)
```

```
## Non-constant Variance Score Test  
## Variance formula: ~ fitted.values  
## Chisquare = 4.360751, Df = 1, p = 0.036776
```

```
cat("\n")
```

```
ncvTest(model_4)
```

```
## Non-constant Variance Score Test  
## Variance formula: ~ fitted.values  
## Chisquare = 4.742532, Df = 1, p = 0.029426
```

```
cat("\n")
```

```
ncvTest(model_5)
```

```
## Non-constant Variance Score Test
## Variance formula: ~ fitted.values
## Chisquare = 12.05193, Df = 1, p = 0.00051739
```

```
cat("\n")
```

```
ncvTest(model_6)
```

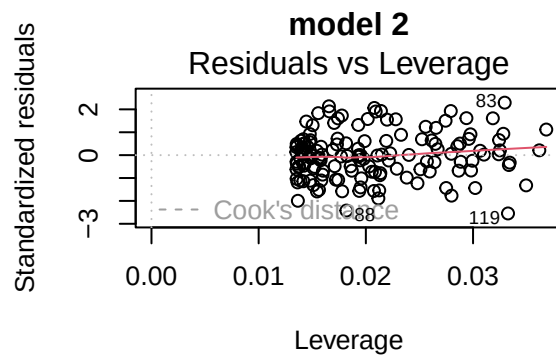
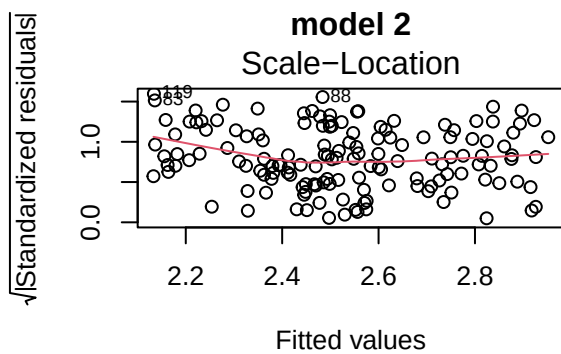
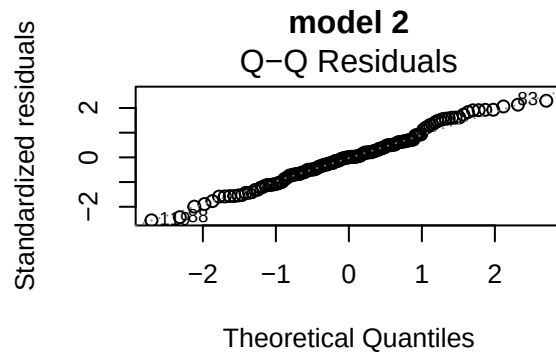
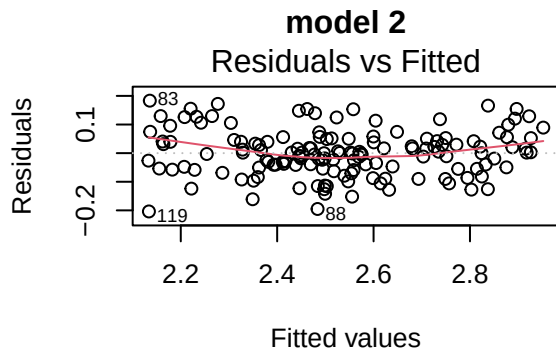
```
## Non-constant Variance Score Test
## Variance formula: ~ fitted.values
## Chisquare = 20.79443, Df = 1, p = 5.1131e-06
```

Ok so same problems I ran into later.

Think we'll remake model_6, using clean_ungulates and transformed density.

```
new.model.1 = lm(log(density) ~ season, data = clean_ungulates)
new.model.2 = lm(log(density) ~ season + ndvi, data = clean_ungulates)
new.model.3 = lm(log(density) ~ season + ndvi + dist_water, data = clean_ungulates)
new.model.4 = lm(log(density) ~ season + ndvi + dist_water + elevation,
                  data = clean_ungulates)
new.model.5 = lm(log(density) ~ season + ndvi + dist_water + elevation
                  + human_footprint, data = clean_ungulates)
new.model.6 = lm(log(density) ~ season + ndvi + dist_water + elevation
                  + human_footprint + snow_depth, data = clean_ungulates)

par(mfrow = c(2,2))
plot(new.model.2, main = "model 2")
```



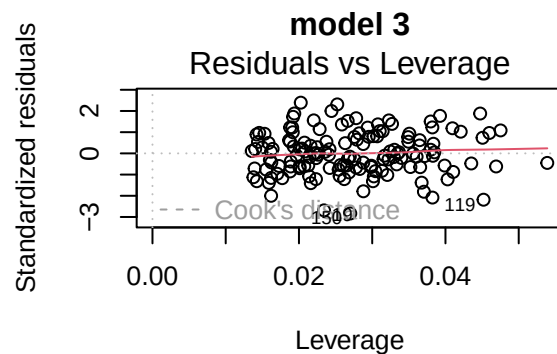
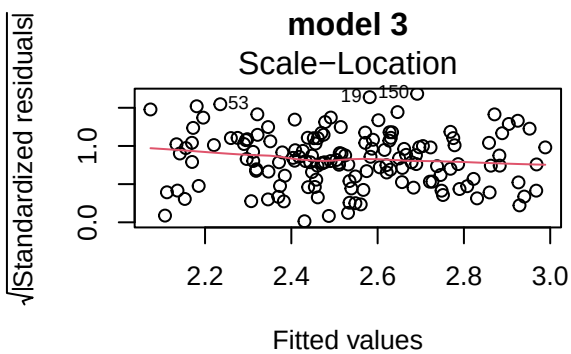
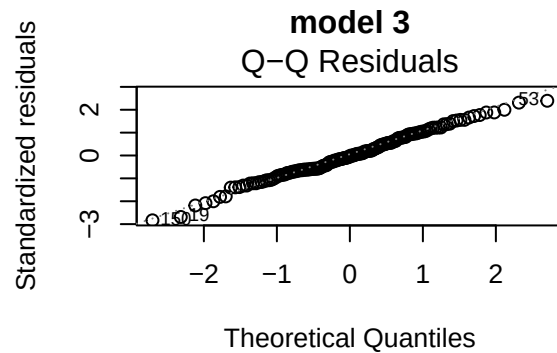
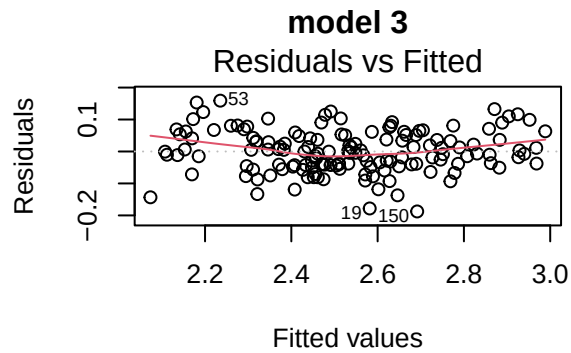
```
shapiro.test(new.model.2$residuals)
```

```
##
##  Shapiro-Wilk normality test
##
## data:  new.model.2$residuals
## W = 0.98846, p-value = 0.2693
```

```
ncvTest(new.model.2)
```

```
## Non-constant Variance Score Test
## Variance formula: ~ fitted.values
## Chisquare = 2.492457, Df = 1, p = 0.11439
```

```
par(mfrow = c(2,2))
plot(new.model.3, main = "model 3")
```



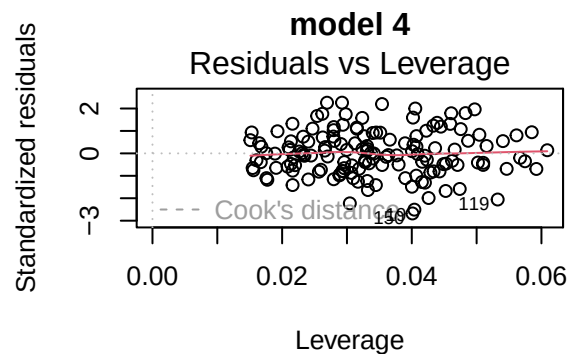
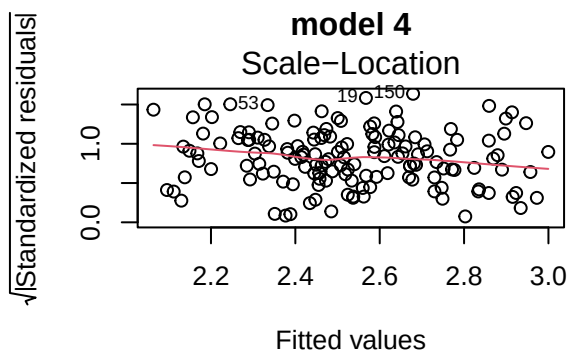
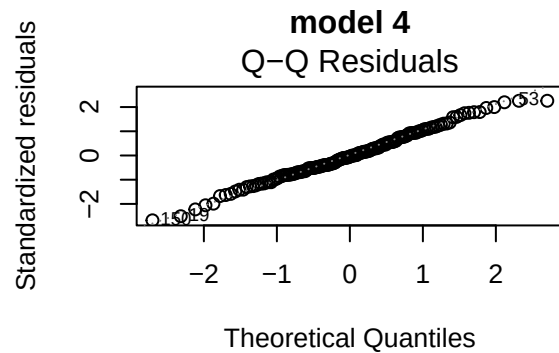
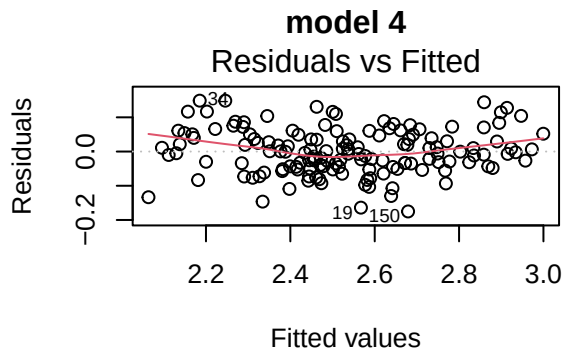
```
shapiro.test(new.model.3$residuals)
```

```
##
##  Shapiro-Wilk normality test
##
## data:  new.model.3$residuals
## W = 0.9931, p-value = 0.7084
```

```
ncvTest(new.model.3)
```

```
## Non-constant Variance Score Test
## Variance formula: ~ fitted.values
## Chisquare = 0.8896152, Df = 1, p = 0.34558
```

```
par(mfrow = c(2,2))
plot(new.model.4, main = "model 4")
```



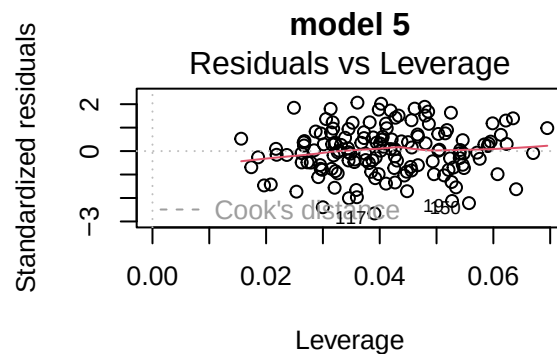
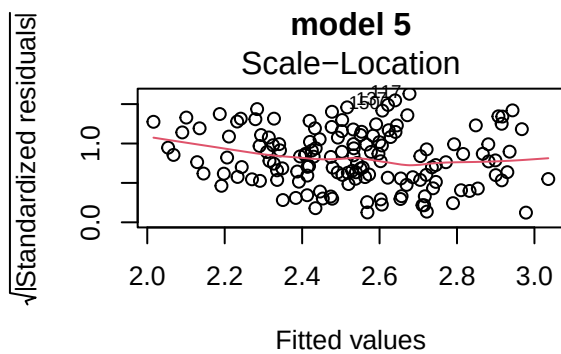
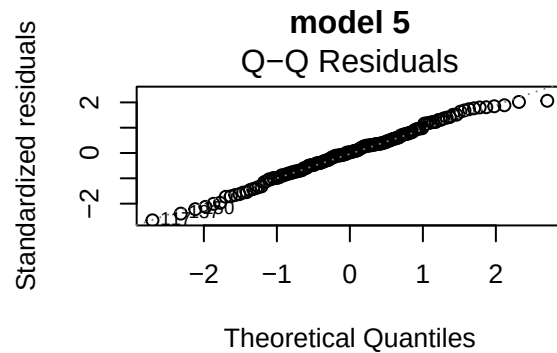
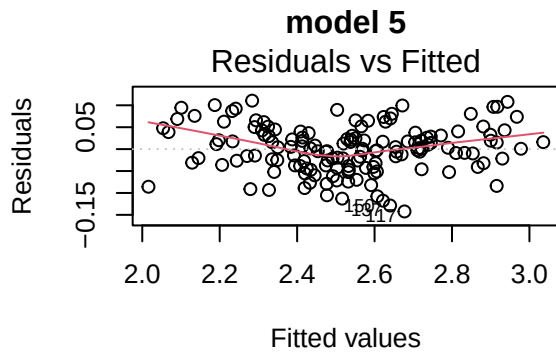
```
shapiro.test(new.model.4$residuals)
```

```
##
##  Shapiro-Wilk normality test
##
## data:  new.model.4$residuals
## W = 0.99359, p-value = 0.7633
```

```
ncvTest(new.model.4)
```

```
## Non-constant Variance Score Test
## Variance formula: ~ fitted.values
## Chisquare = 0.9354059, Df = 1, p = 0.33346
```

```
par(mfrow = c(2,2))
plot(new.model.5, main = "model 5")
```



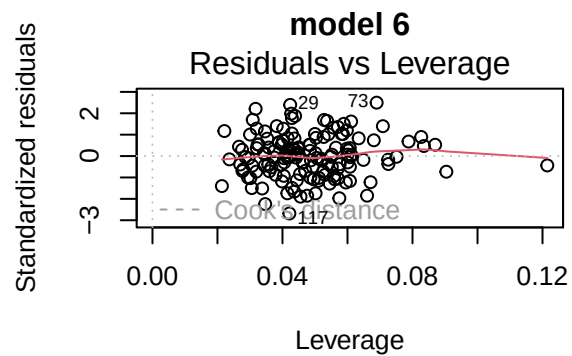
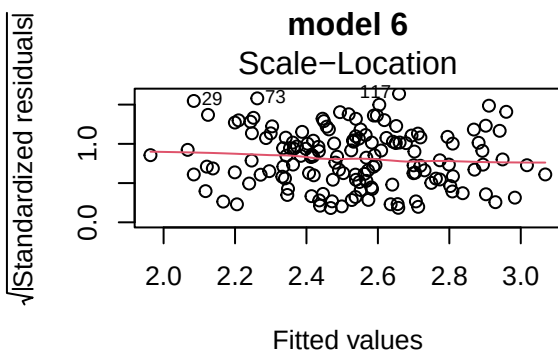
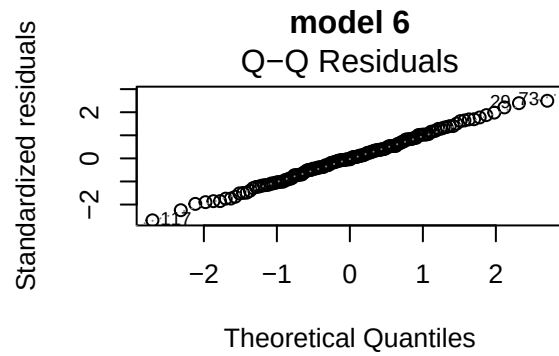
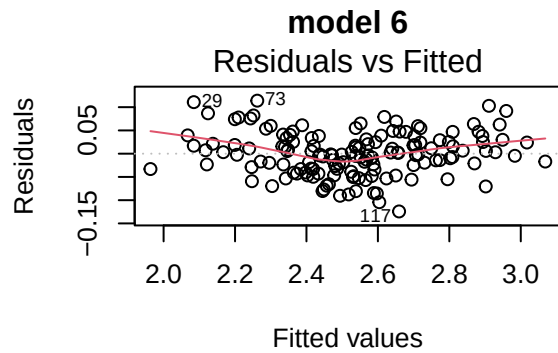
```
shapiro.test(new.model.5$residuals)
```

```
##
##  Shapiro-Wilk normality test
##
## data:  new.model.5$residuals
## W = 0.99098, p-value = 0.4769
```

```
ncvTest(new.model.5)
```

```
## Non-constant Variance Score Test
## Variance formula: ~ fitted.values
## Chisquare = 0.1948711, Df = 1, p = 0.65889
```

```
par(mfrow = c(2,2))
plot(new.model.6, main = "model 6")
```



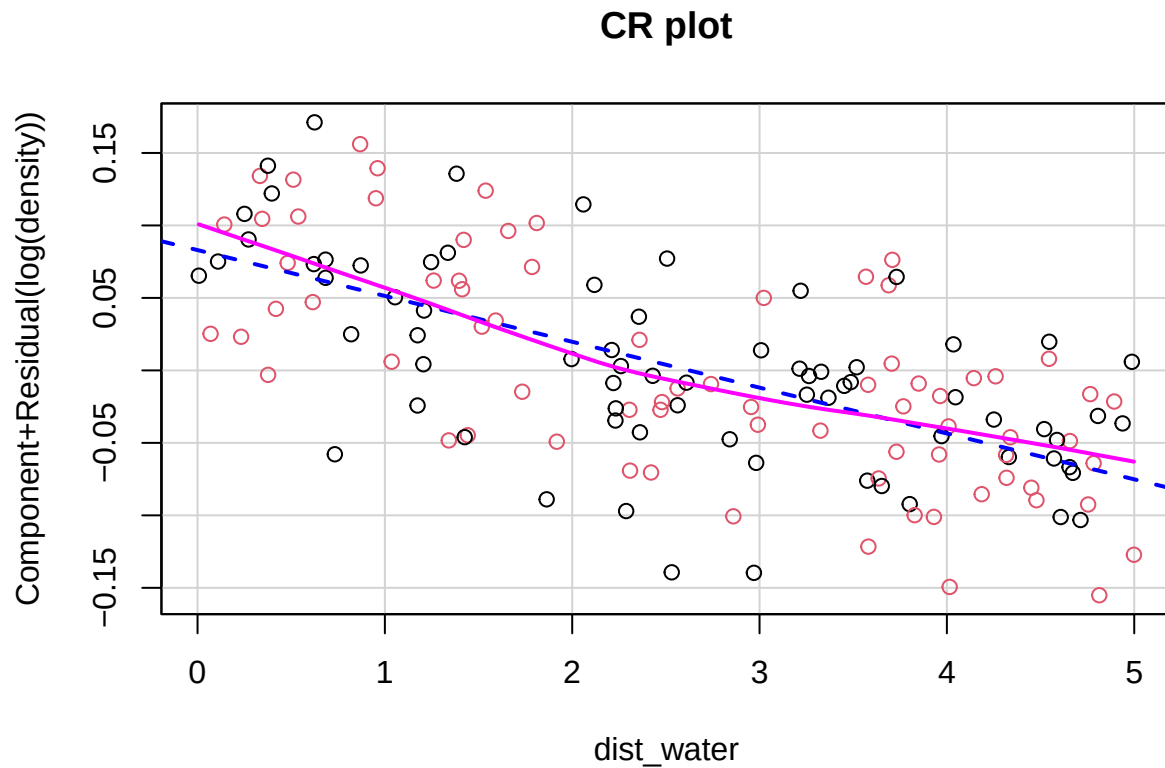
```
shapiro.test(new.model.6$residuals)
```

```
##
##  Shapiro-Wilk normality test
##
## data:  new.model.6$residuals
## W = 0.99676, p-value = 0.9882
```

```
ncvTest(new.model.6)
```

```
## Non-constant Variance Score Test
## Variance formula: ~ fitted.values
## Chisquare = 0.6246597, Df = 1, p = 0.42932
```

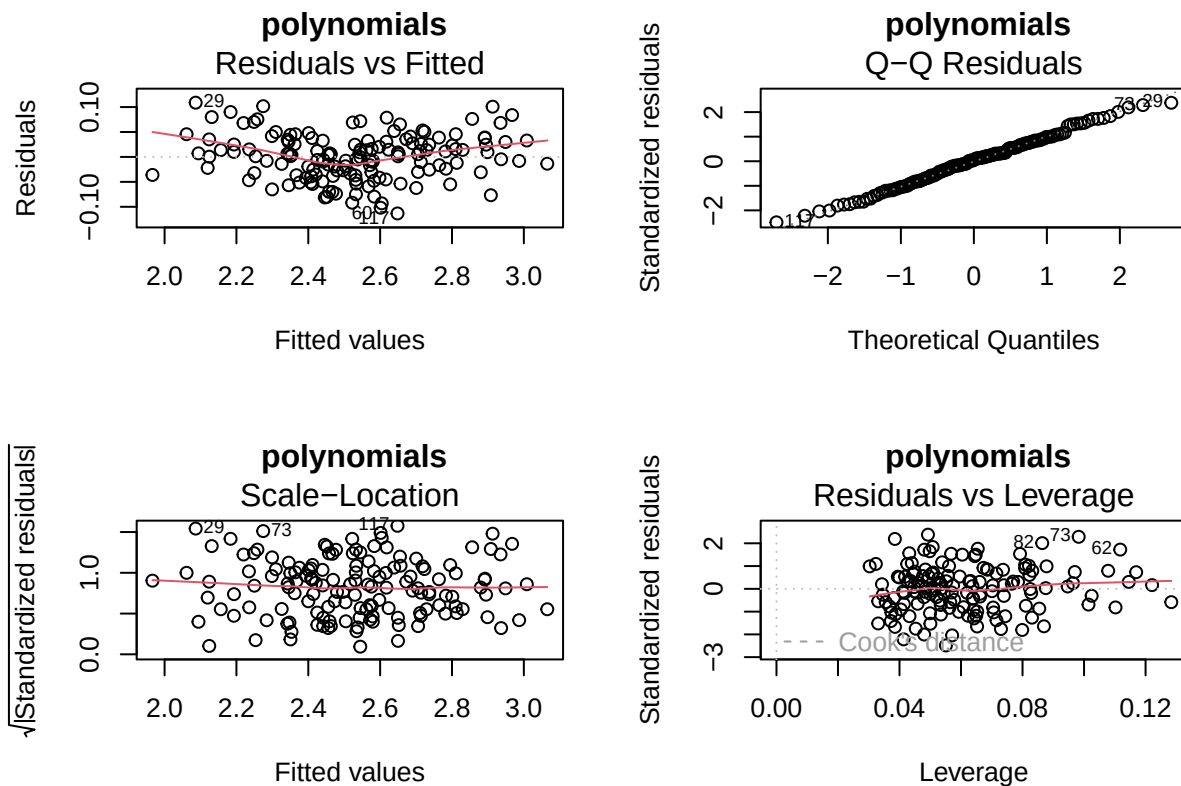
```
# checking linearity
crPlot(model = new.model.5, variable = "dist_water", main = "CR plot",
       col = clean_ungulates$season)
```



Having gone through the diagnostics and also plotted all the crPlot combos of predictor to model, the diagnostic stats suggest everything is fine for variance and normality, despite the plots looking a little wonky. The linearity seems intrinsically tied to the season. However, `dist_water` in `new.model.5` and `elevation` all throughout seem to have some non-linear tendencies.

```
new.poly.model = lm(log(density) ~ season + ndvi + poly(dist_water, 2, raw = TRUE)
                    + poly(elevation, 2, raw = TRUE) + human_footprint + snow_depth,
                    data = clean_ungulates)

par(mfrow = c(2,2))
plot(new.poly.model, main = " polynomials")
```



```
shapiro.test(new.poly.model$residuals)
```

```
##
##  Shapiro-Wilk normality test
##
## data:  new.poly.model$residuals
## W = 0.99471, p-value = 0.875
```

```
ncvTest(new.poly.model)
```

```
## Non-constant Variance Score Test
## Variance formula: ~ fitted.values
## Chisquare = 0.2552156, Df = 1, p = 0.61343
```

That doesn't really solve it. So think its due to seasons itself, will leave it for now.

```
summary(new.model.1)
```

```
##
## Call:
## lm(formula = log(density) ~ season, data = clean_ungulates)
##
## Residuals:
```

```
##      Min      1Q   Median      3Q      Max
## -0.43914 -0.12811  0.00903  0.09788  0.35331
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   2.69834    0.01902  141.86  <2e-16 ***
## seasonWinter -0.32902    0.02672  -12.31  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.1614 on 144 degrees of freedom
## Multiple R-squared:  0.5129, Adjusted R-squared:  0.5096
## F-statistic: 151.7 on 1 and 144 DF,  p-value: < 2.2e-16
```

```
summary(new.model.2)
```

```
##
## Call:
## lm(formula = log(density) ~ season + ndvi, data = clean_ungulates)
##
## Residuals:
##      Min      1Q   Median      3Q      Max
## -0.203748 -0.052990  0.000261  0.048455  0.183322
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   2.32283    0.02059  112.79  <2e-16 ***
## seasonWinter -0.33989    0.01347  -25.23  <2e-16 ***
## ndvi          0.71381    0.03465   20.60  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.08131 on 143 degrees of freedom
## Multiple R-squared:  0.8772, Adjusted R-squared:  0.8755
## F-statistic: 510.9 on 2 and 143 DF,  p-value: < 2.2e-16
```

```
summary(new.model.3)
```

```
##
## Call:
## lm(formula = log(density) ~ season + ndvi + dist_water, data = clean_ungulates)
##
## Residuals:
##      Min      1Q   Median      3Q      Max
## -0.18761 -0.04250 -0.00234  0.05005  0.15897
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   2.413815    0.020279  119.032  < 2e-16 ***
## seasonWinter -0.336550    0.011129  -30.240  < 2e-16 ***
## ndvi          0.693899    0.028711   24.168  < 2e-16 ***
## dist_water    -0.031266    0.003798   -8.233 1.08e-13 ***
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.06713 on 142 degrees of freedom
## Multiple R-squared:  0.9169, Adjusted R-squared:  0.9151
## F-statistic: 522.3 on 3 and 142 DF,  p-value: < 2.2e-16
```

```
summary(new.model.4)
```

```
##
## Call:
## lm(formula = log(density) ~ season + ndvi + dist_water + elevation,
##     data = clean_ungulates)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.175063 -0.044378 -0.003059  0.048446  0.148642
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   2.435e+00  2.392e-02 101.816 < 2e-16 ***
## seasonWinter -3.385e-01  1.113e-02 -30.421 < 2e-16 ***
## ndvi          6.906e-01  2.861e-02  24.137 < 2e-16 ***
## dist_water    -3.073e-02  3.789e-03  -8.111 2.2e-13 ***
## elevation     -1.452e-05  8.838e-06  -1.643  0.103
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.06674 on 141 degrees of freedom
## Multiple R-squared:  0.9185, Adjusted R-squared:  0.9162
## F-statistic: 397.1 on 4 and 141 DF,  p-value: < 2.2e-16
```

```
summary(new.model.5)
```

```
##
## Call:
## lm(formula = log(density) ~ season + ndvi + dist_water + elevation +
##     human_footprint, data = clean_ungulates)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.142231 -0.036852  0.001355  0.032789  0.110585
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   2.513e+00  2.163e-02 116.172 < 2e-16 ***
## seasonWinter  -3.406e-01  9.103e-03 -37.418 < 2e-16 ***
## ndvi          6.762e-01  2.346e-02  28.827 < 2e-16 ***
## dist_water    -3.163e-02  3.100e-03 -10.202 < 2e-16 ***
## elevation     -1.754e-05  7.236e-06  -2.424  0.0166 *
## human_footprint -1.226e-03  1.457e-04  -8.418 4.05e-14 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
```

```
## Residual standard error: 0.05457 on 140 degrees of freedom
## Multiple R-squared: 0.9459, Adjusted R-squared: 0.9439
## F-statistic: 489.2 on 5 and 140 DF, p-value: < 2.2e-16
```

```
summary(new.model.6)
```

```
##
## Call:
## lm(formula = log(density) ~ season + ndvi + dist_water + elevation +
##     human_footprint + snow_depth, data = clean_ungulates)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.124250 -0.032431 -0.001646  0.029594  0.113980
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   2.675e+00  3.014e-02  88.745 < 2e-16 ***
## seasonWinter  -3.413e-01  7.895e-03 -43.230 < 2e-16 ***
## ndvi           5.405e-01  2.837e-02  19.051 < 2e-16 ***
## dist_water    -3.100e-02  2.690e-03 -11.522 < 2e-16 ***
## elevation     -2.170e-05  6.305e-06  -3.442 0.000764 ***
## human_footprint -1.256e-03  1.264e-04  -9.935 < 2e-16 ***
## snow_depth    -2.890e-03  4.209e-04  -6.865 2.02e-10 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.04733 on 139 degrees of freedom
## Multiple R-squared: 0.9596, Adjusted R-squared: 0.9578
## F-statistic: 549.9 on 6 and 139 DF, p-value: < 2.2e-16
```

Now lets do lrtests on these:

```
#lrtest new 1 v 2
lrtest(new.model.1, new.model.2)
```

```
## Likelihood ratio test
##
## Model 1: log(density) ~ season
## Model 2: log(density) ~ season + ndvi
##   #Df LogLik Df  Chisq Pr(>Chisq)
## 1    3  60.125
## 2    4 160.730  1 201.21 < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

new.model.2 more likely

```
# lrtest new 2 v 3
lrtest(new.model.2, new.model.3)
```

```
## Likelihood ratio test
```

```
##
## Model 1: log(density) ~ season + ndvi
## Model 2: log(density) ~ season + ndvi + dist_water
##   #Df LogLik Df  Chisq Pr(>Chisq)
## 1    4 160.73
## 2    5 189.22  1 56.972  4.419e-14 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

new.model.3 more likely

```
# lrtest new 3 v 4
lrtest(new.model.3, new.model.4)
```

```
## Likelihood ratio test
##
## Model 1: log(density) ~ season + ndvi + dist_water
## Model 2: log(density) ~ season + ndvi + dist_water + elevation
##   #Df LogLik Df  Chisq Pr(>Chisq)
## 1    5 189.22
## 2    6 190.60  1 2.7691    0.0961 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

not enough improvement, new.model.3 still more likely

```
# lrtest new 3 v 5
lrtest(new.model.3, new.model.5)
```

```
## Likelihood ratio test
##
## Model 1: log(density) ~ season + ndvi + dist_water
## Model 2: log(density) ~ season + ndvi + dist_water + elevation + human_footprint
##   #Df LogLik Df  Chisq Pr(>Chisq)
## 1    5 189.22
## 2    7 220.50  2 62.568  2.592e-14 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

new.model.5 more likely

```
# lrtest new 5 v 6
lrtest(new.model.5, new.model.6)
```

```
## Likelihood ratio test
##
## Model 1: log(density) ~ season + ndvi + dist_water + elevation + human_footprint
## Model 2: log(density) ~ season + ndvi + dist_water + elevation + human_footprint +
##          snow_depth
##   #Df LogLik Df  Chisq Pr(>Chisq)
## 1    7 220.50
## 2    8 241.82  1 42.63  6.612e-11 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

new.model.6 is an improvement, more likely.

So, having done all of that again, compare new.model.6 with model.6.v3

```
# lrtest new.model.6 v model.6.v3
lrtest(new.model.6, model.6.v3)

## Likelihood ratio test
##
## Model 1: log(density) ~ season + ndvi + dist_water + elevation + human_footprint +
##      snow_depth
## Model 2: log(density) ~ season * (ndvi + dist_water + elevation + human_footprint +
##      snow_depth)
##      #Df LogLik Df  Chisq Pr(>Chisq)
## 1      8 241.82
## 2     13 261.01  5  38.395  3.144e-07 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

model.6.v3 has 5 more degrees of freedom due to the interactions, but has a higher likelihood.

```
AICc(new.model.3, new.model.4)
```

```
##           df      AICc
## new.model.3  5 -368.0044
## new.model.4  6 -368.5978
```

```
AICc(new.model.3, new.model.5)
```

```
##           df      AICc
## new.model.3  5 -368.0044
## new.model.5  7 -426.1891
```

```
AICc(new.model.5, new.model.6)
```

```
##           df      AICc
## new.model.5  7 -426.1891
## new.model.6  8 -466.5800
```

```
AICc(new.model.6, model.6.v3)
```

```
##           df      AICc
## new.model.6  8 -466.5800
## model.6.v3   13 -493.2681
```

So using lrtest() and AICc, can confirm that the best model for the density of ungulates used the main and interaction effects between each of the continuous predictors and season.

Final Model and details

Firstly, should be noted that the following points were removed from the dataset (for making the models) in order to achieve the assumptions of linearity, normality, and homogeneity:

```
removed_points = full_ungulates[which(CD.model.6 > CD.crit), ]
removed_points
```

```
##          ndvi dist_water elevation human_footprint snow_depth density region
## 39  0.8254298   1.868626 1807.2166         50.39939   8.197178 16.12204  North
## 61  0.3733503   4.613136 2213.6940         89.73185  45.934787 18.47730  South
## 90  0.8097978   4.411319  383.4791         74.64486  34.598064 17.62848  Central
## 130 0.7840584   1.587397  988.2534         72.74659   9.559316 19.25493  North
##      season
## 39  Winter
## 61  Winter
## 90  Winter
## 130 Winter
```

The best model from dredge() was best_model\$576.

```
# putting the best model, 576, in a table
```

```
final_model = best_model$`576`
best_model_summary = summary(best_model$`576`)
final_model_estimates = as.data.frame(best_model_summary$coefficients)
final_model_estimates
```

```
##              Estimate Std. Error  t value    Pr(>|t|)
## (Intercept)  2.622044e+00 2.802476e-02  93.561702 1.013810e-126
## dist_water   -3.121688e-02 2.385380e-03 -13.086751 5.971765e-26
## elevation    -2.102015e-05 5.590754e-06  -3.759805 2.503171e-04
## human_footprint -1.276957e-03 1.121229e-04 -11.388903 1.344139e-21
## ndvi          6.432322e-01 3.007416e-02  21.388201 1.187425e-45
## seasonWinter  -2.217951e-01 2.041602e-02 -10.863777 2.994895e-20
## snow_depth    -2.913791e-03 3.731685e-04  -7.808245 1.308584e-12
## ndvi:seasonWinter -2.235919e-01 3.587702e-02  -6.232177 5.230637e-09
```

```
# reporting the adjusted R^2 and p-value for the final model
```

```
#final_model_sig = best_model_summary$adj.r.squared
final_model_sig = data.frame("adjR2" = best_model_summary$adj.r.squared,
                             "AICc" = models[1, "AICc"],
                             row.names = "final model")
final_model_sig
```

```
##          adjR2      AICc
## final model 0.9668527 -500.5144
```

Think this is about what I need for statistics.

Making these cleaner for slides

```
model.1.v3 = lm(log(density) ~ season, data = clean_ungulates)
model.2.v3 = lm(log(density) ~ season * ndvi, data = clean_ungulates)
```

```

model.3.v3 = lm(log(density) ~ season * (ndvi + dist_water),
                data = clean_ungulates)
model.4.v3 = lm(log(density) ~ season * (ndvi + dist_water + elevation),
                data = clean_ungulates)
model.5.v3 = lm(log(density) ~ season * (ndvi + dist_water + elevation
                + human_footprint),
                data = clean_ungulates)
model.6.v3 = lm(log(density) ~ season *
                (ndvi + dist_water + elevation + human_footprint + snow_depth),
                data = clean_ungulates)

```

```

# Shapiro-Wilks and ncvTest
shapiro.test(model.2.v3$residuals)

```

```

##
##  Shapiro-Wilk normality test
##
## data:  model.2.v3$residuals
## W = 0.99041, p-value = 0.4227

```

```

ncvTest(model.2.v3)

```

```

## Non-constant Variance Score Test
## Variance formula: ~ fitted.values
## Chisquare = 2.738826, Df = 1, p = 0.097937

```

```

shapiro.test(model.3.v3$residuals)

```

```

##
##  Shapiro-Wilk normality test
##
## data:  model.3.v3$residuals
## W = 0.99155, p-value = 0.5361

```

```

ncvTest(model.3.v3)

```

```

## Non-constant Variance Score Test
## Variance formula: ~ fitted.values
## Chisquare = 1.237171, Df = 1, p = 0.26602

```

```

shapiro.test(model.4.v3$residuals)

```

```

##
##  Shapiro-Wilk normality test
##
## data:  model.4.v3$residuals
## W = 0.99218, p-value = 0.6046

```

```
ncvTest(model.4.v3)
```

```
## Non-constant Variance Score Test
## Variance formula: ~ fitted.values
## Chisquare = 1.249877, Df = 1, p = 0.26358
```

```
shapiro.test(model.5.v3$residuals)
```

```
##
## Shapiro-Wilk normality test
##
## data: model.5.v3$residuals
## W = 0.98547, p-value = 0.1278
```

```
ncvTest(model.5.v3)
```

```
## Non-constant Variance Score Test
## Variance formula: ~ fitted.values
## Chisquare = 0.01313613, Df = 1, p = 0.90875
```

```
shapiro.test(model.6.v3$residuals)
```

```
##
## Shapiro-Wilk normality test
##
## data: model.6.v3$residuals
## W = 0.99501, p-value = 0.9002
```

```
ncvTest(model.6.v3)
```

```
## Non-constant Variance Score Test
## Variance formula: ~ fitted.values
## Chisquare = 0.1825325, Df = 1, p = 0.66921
```

```
summary(model.1.v3)
```

```
##
## Call:
## lm(formula = log(density) ~ season, data = clean_ungulates)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.43914 -0.12811  0.00903  0.09788  0.35331
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   2.69834    0.01902  141.86  <2e-16 ***
## seasonWinter -0.32902    0.02672  -12.31  <2e-16 ***
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.1614 on 144 degrees of freedom
## Multiple R-squared:  0.5129, Adjusted R-squared:  0.5096
## F-statistic: 151.7 on 1 and 144 DF,  p-value: < 2.2e-16
```

```
summary(model.2.v3)
```

```
##
## Call:
## lm(formula = log(density) ~ season * ndvi, data = clean_ungulates)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.239875 -0.052504  0.004327  0.044685  0.172095
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      2.27265     0.02598   87.492 < 2e-16 ***
## seasonWinter     -0.23041     0.03839   -6.001 1.55e-08 ***
## ndvi              0.80920     0.04609   17.557 < 2e-16 ***
## seasonWinter:ndvi -0.20495     0.06756   -3.034 0.00287 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.07908 on 142 degrees of freedom
## Multiple R-squared:  0.8847, Adjusted R-squared:  0.8823
## F-statistic: 363.2 on 3 and 142 DF,  p-value: < 2.2e-16
```

```
summary(model.3.v3)
```

```
##
## Call:
## lm(formula = log(density) ~ season * (ndvi + dist_water), data = clean_ungulates)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.173549 -0.039985  0.001301  0.040924  0.145895
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      2.351456     0.025928   90.691 < 2e-16 ***
## seasonWinter     -0.201982     0.037830   -5.339 3.68e-07 ***
## ndvi              0.794048     0.037463   21.196 < 2e-16 ***
## dist_water       -0.027509     0.005285   -5.205 6.76e-07 ***
## seasonWinter:ndvi -0.215479     0.054945   -3.922 0.000137 ***
## seasonWinter:dist_water -0.007419     0.007263   -1.021 0.308816
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.06408 on 140 degrees of freedom
## Multiple R-squared:  0.9254, Adjusted R-squared:  0.9227
## F-statistic: 347.1 on 5 and 140 DF,  p-value: < 2.2e-16
```

```
summary(model.4.v3)
```

```
##
## Call:
## lm(formula = log(density) ~ season * (ndvi + dist_water + elevation),
##     data = clean_ungulates)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.166168 -0.042954 -0.002207  0.042997  0.138431
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      2.377e+00  3.138e-02  75.744 < 2e-16 ***
## seasonWinter     -2.138e-01  4.444e-02  -4.811 3.89e-06 ***
## ndvi              7.891e-01  3.752e-02  21.030 < 2e-16 ***
## dist_water       -2.692e-02  5.286e-03  -5.094 1.13e-06 ***
## elevation        -1.688e-05  1.188e-05  -1.421 0.157703
## seasonWinter:ndvi -2.121e-01  5.493e-02  -3.861 0.000173 ***
## seasonWinter:dist_water -7.596e-03  7.269e-03  -1.045 0.297880
## seasonWinter:elevation  6.315e-06  1.693e-05   0.373 0.709749
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.0639 on 138 degrees of freedom
## Multiple R-squared:  0.9268, Adjusted R-squared:  0.9231
## F-statistic: 249.7 on 7 and 138 DF,  p-value: < 2.2e-16
```

```
summary(model.5.v3)
```

```
##
## Call:
## lm(formula = log(density) ~ season * (ndvi + dist_water + elevation +
##     human_footprint), data = clean_ungulates)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.129219 -0.026279  0.002869  0.034297  0.101393
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      2.442e+00  2.710e-02  90.120 < 2e-16 ***
## seasonWinter     -1.884e-01  3.878e-02  -4.857 3.23e-06 ***
## ndvi              7.815e-01  2.953e-02  26.464 < 2e-16 ***
## dist_water       -2.753e-02  4.157e-03  -6.622 7.56e-10 ***
## elevation        -1.814e-05  9.343e-06  -1.941  0.0543 .
## human_footprint   -1.126e-03  1.921e-04  -5.862 3.30e-08 ***
## seasonWinter:ndvi -2.279e-01  4.333e-02  -5.260 5.46e-07 ***
## seasonWinter:dist_water -8.226e-03  5.719e-03  -1.438  0.1526
## seasonWinter:elevation  2.437e-06  1.333e-05   0.183  0.8552
## seasonWinter:human_footprint -2.386e-04  2.685e-04  -0.889  0.3758
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.05024 on 136 degrees of freedom
## Multiple R-squared:  0.9554, Adjusted R-squared:  0.9525
## F-statistic: 323.9 on 9 and 136 DF,  p-value: < 2.2e-16
```

```
summary(model.6.v3)
```

```
##
## Call:
## lm(formula = log(density) ~ season * (ndvi + dist_water + elevation +
##     human_footprint + snow_depth), data = clean_ungulates)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.110075 -0.024035  0.001402  0.029434  0.126154
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      2.615e+00  3.820e-02  68.466 < 2e-16 ***
## seasonWinter     -2.122e-01  5.338e-02  -3.975 0.000114 ***
## ndvi              6.403e-01  3.525e-02  18.166 < 2e-16 ***
## dist_water       -2.864e-02  3.503e-03  -8.176 1.97e-13 ***
## elevation        -2.168e-05  7.885e-06  -2.750 0.006779 **
## human_footprint  -1.173e-03  1.618e-04  -7.247 3.04e-11 ***
## snow_depth       -3.017e-03  5.345e-04  -5.644 9.50e-08 ***
## seasonWinter:ndvi -2.155e-01  5.083e-02  -4.240 4.13e-05 ***
## seasonWinter:dist_water -5.090e-03  4.831e-03  -1.054 0.293926
## seasonWinter:elevation  1.319e-06  1.127e-05   0.117 0.906998
## seasonWinter:human_footprint -2.063e-04  2.261e-04  -0.913 0.363049
## seasonWinter:snow_depth  2.830e-04  7.547e-04   0.375 0.708243
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.04226 on 134 degrees of freedom
## Multiple R-squared:  0.9689, Adjusted R-squared:  0.9664
## F-statistic: 379.8 on 11 and 134 DF,  p-value: < 2.2e-16
```

```
AICc(model.3.v3, model.4.v3)
```

```
##           df      AICc
## model.3.v3  7 -379.2902
## model.4.v3  9 -377.6947
```

```
AICc(model.3.v3, model.5.v3)
```

```
##           df      AICc
## model.3.v3  7 -379.2902
## model.5.v3 11 -445.4091
```

```
AICc(model.5.v3, model.6.v3)
```

```
##           df      AICc  
## model.5.v3 11 -445.4091  
## model.6.v3 13 -493.2681
```

```
best_aicc = AICc(model.6.v3)  
second_aicc = AICc(new.model.6)
```

```
ER = exp((second_aicc - best_aicc) / 2)  
ER
```

```
## [1] 624088.2
```